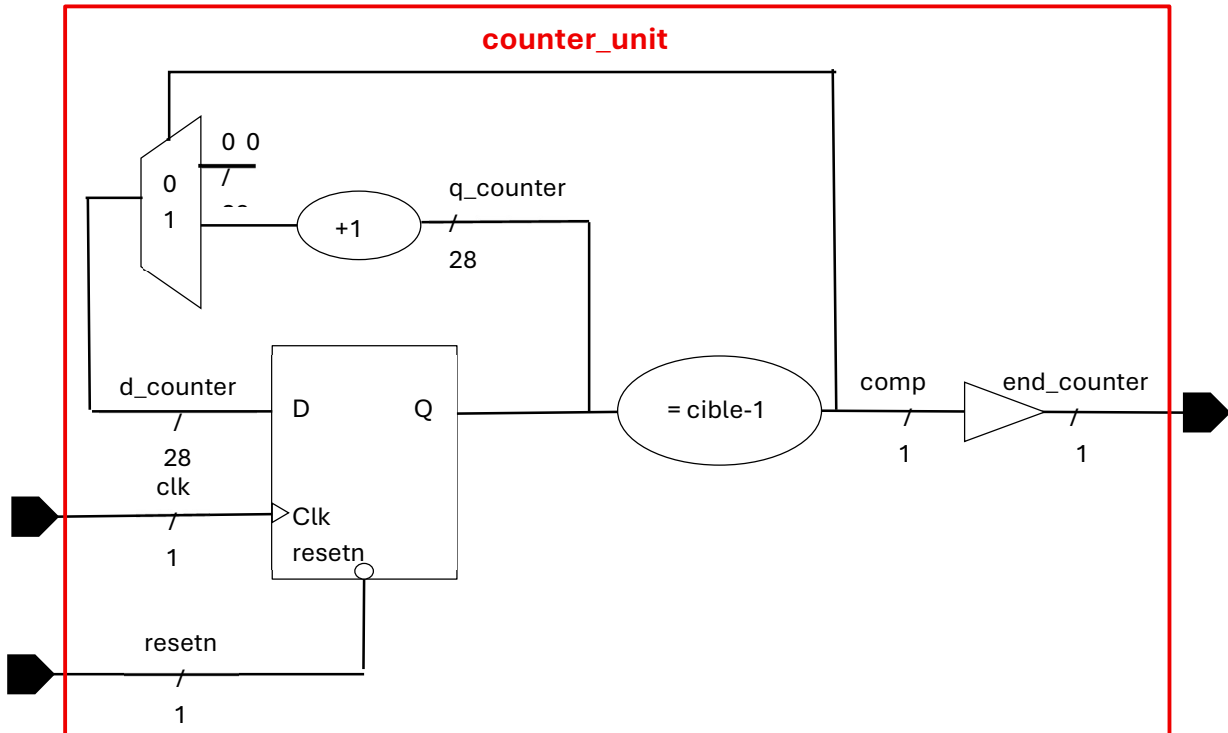


TP3 – Machine à états (FSM)

- Dans un fichier .vhd, créez un module Counter_unit à partir du compteur du TP1. Le module prendra en entrée un signal d'horloge et de resetn, et donnera en sortie le signal end_counter. Utilisez un paramètre generic() pour définir le nombre de coup d'horloge à compter.



Le module counter_unit est composé d'un registre 28 bits dont la sortie s'incrémente à chaque coup d'horloge. Cette sortie *q_counter* est ensuite comparée à une valeur *cible-1* qui sera déclarée en paramètre generic. On compare à cible-1 car si on veut cible coups d'horloge et que le compteur commence à compter à 0 : il faut arrêter le comptage à cible - 1

La sortie end_counter passera à 1 sur un coup d'horloge à chaque fin de comptage.

Au front d'horloge suivant le compteur se réinitialise à 0, un nouveau cycle de comptage recommence.

Le sortie end_counter sera donc une impulsion d'un coup d'horloge à chaque fin de comptage.

J'ai rajouté un buffer sur la sortie pour pouvoir utiliser le résultat de la comparaison comme entrée du multiplexeur.

```

library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

entity counter_unit is
    generic (
        cible : integer := 20
    );
    port ( clk : in STD_LOGIC;
          resetn : in STD_LOGIC;
          end_counter : out STD_LOGIC
        );
end counter_unit;

architecture Behavioral of counter_unit is
    signal d_counter : std_logic_vector ( 27 downto 0) := (others => '0');
    signal q_counter : std_logic_vector ( 27 downto 0) := (others => '0');
    signal comp : std_logic := '0'; -- sortie du comparateur

    begin
        process(clk, resetn)
        begin
            if (resetn = '0') then
                q_counter <= (others => '0');
            elsif(rising_edge(clk)) then
                q_counter <= d_counter;
            end if;
        end process;

        comp <= '1' when to_integer(unsigned(q_counter)) = cible -1 else '0'; -- on valide le comptage à cible -1
        d_counter <= q_counter + 1 when comp = '0' else (others => '0'); -- on remet le registre à 0 quand on a atteint la cible
        end_counter <= comp; -- rajout d'un buffer sur la sortie
    end Behavioral;

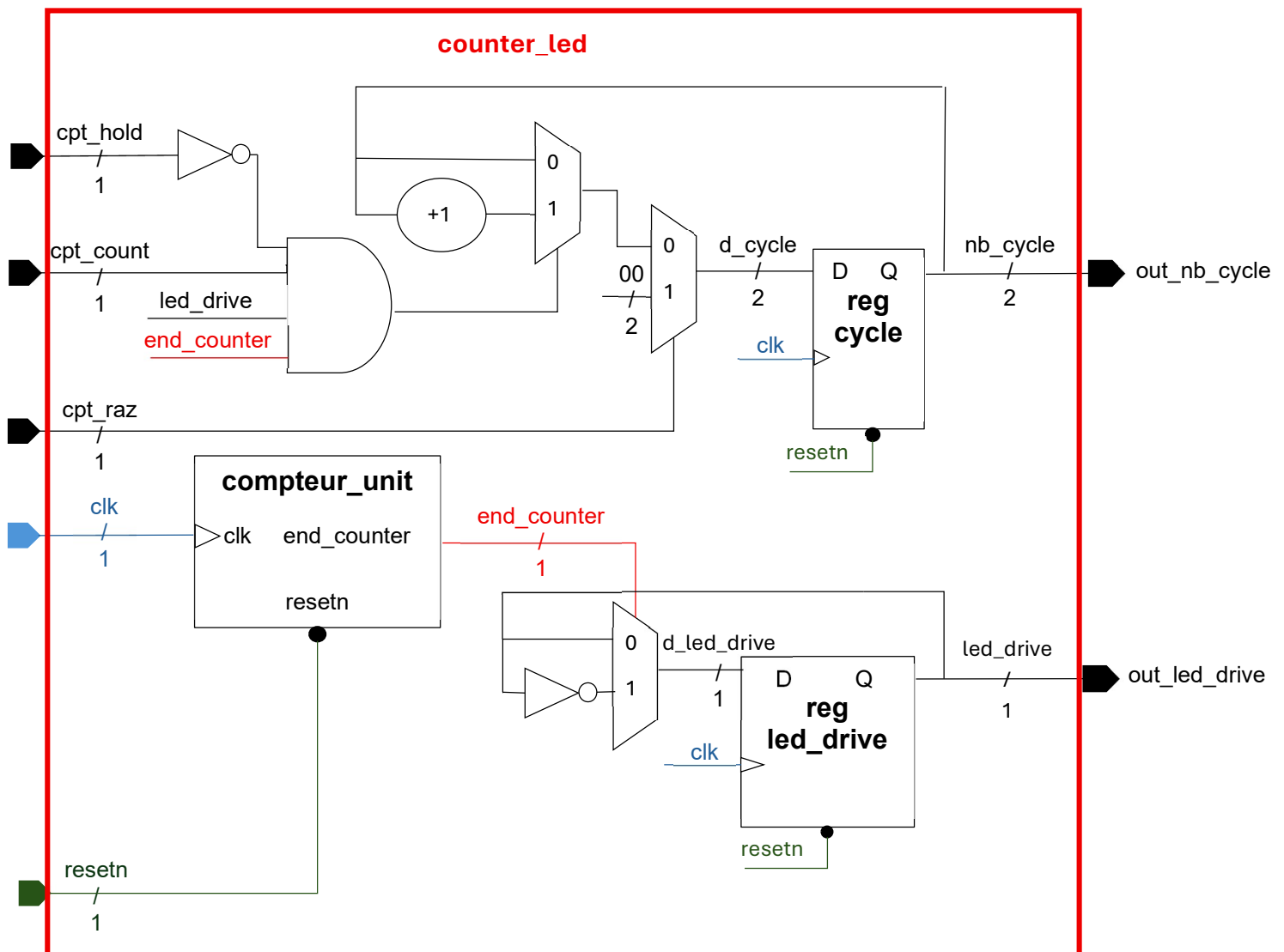
```

Le process séquentiel contient uniquement le comportement du registre 28 bits.

Le comportement des multiplexeurs, additionneur et comparateur est décrit en combinatoire

2. En schéma RTL, créez un compteur du signal end_counter. Ce compteur doit permettre de déterminer le nombre de cycles allumé/éteint qui ont été effectués par la LED. Le compteur doit pouvoir être remis à 0, maintenir sa valeur actuelle ou s'incrémenter.

- Les entrées
 - clk : signal d'horloge de la carte
 - resetn : resetn général de la carte
 - cpt_raz : signal de remise à zéro du compteur de cycle
 - cpt_hold : signal de maintien de la sortie du compteur de cycle
 - cpt_count : signal d'autorisation de comptage du compteur de cycle
- Les sorties
 - out_nb_cycle : sortie du compteur de cycle
 - out_led_drive : sortie qui pilote l'allumage des LED



- Le bus de sortie nb_cycle est défini sur 2 bits car par la suite il faudra compter uniquement 3 cycles pour allumer les LED.
- Rajout d'un signal led_drive pour contrôler l'allumage des LED aux étapes suivantes : le signal led_drive change de niveau à chaque fin de comptage du compteur unitaire (end_counter = '1'). Le signal led_drive sera remis à zéro les d'un resetn. Il ne sera pas impacté par les entrées cpt_raz, cpt_hold, cpt_count
- Le signal resetn remet à zéro tous les registres de compteur_unit, registre led_drive et registre cycle

- Table de vérité pour calculer le signal d_cycle : comptage des cycles d'allumage des LED
 - La remise à zéro (cpt_raz) est prioritaire : le compteur est remis à zéro quel que soit l'état des autres signaux
 - Le maintien de la valeur (cpt_hold) roulera la sortie du registre directement sur l'entrée pour garder la valeur
 - L'incrément (cpt_count) roulera la sortie du registre incrémentée de 1 sur l'entrée quand le compteur unitaire aura fini son cycle (end_counter = '1') et quand la LED sera allumée (q_led_drive = '1')

cpt_raz	cpt_count	cpt_hold	end_counter	led_drive	d_cycle	note
1	#	#	#	#	0	raz prioritaire
0	0	0	0	0	q_cycle	
0	#	1	#	#	q_cycle	hold garde la valeur
0	1	0	0	0	q_cycle	
0	1	0	0	1	q_cycle	
0	1	0	1	0	q_cycle	
0	1	0	1	1	q_cycle+1	count incremente

Formulation de d_cycle :

```
d_cycle <= (others => '0') when cpt_raz = '1' else
           nb_cycle + 1 when cpt_count = '1' and end_counter = '1' and cpt_hold = '0' and led_drive = '1' else
           nb_cycle ;
```

3. Ecrivez un code VHDL décrivant ce compteur de cycle, vous utiliserez le module Counter_unit.

```
library ieee;
use ieee.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

entity counter_led is
  generic (
    cible : integer := 20
  );
  Port ( clk      : in std_logic;
        resetn   : in std_logic;
        cpt_raz   : in std_logic;
        cpt_hold  : in std_logic;
        cpt_count : in std_logic;
        out_nb_cycle : out std_logic_vector(1 downto 0) := (others => '0');
        out_led_drive : out std_logic
      );
end counter_led;
```

```

architecture Behavioral of counter_led is
    signal end_counter : std_logic := '0';
    signal led_drive : std_logic := '1';
    signal d_led_drive : std_logic := '1';
    signal nb_cycle : std_logic_vector(1 downto 0) := (others => '0');
    signal d_cycle : std_logic_vector(1 downto 0) := (others => '0');
begin
    cpt: entity work.counter_unit(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clk,
            resetn => resetn,
            end_counter => end_counter
        );
    process(clk, resetn)
    begin
        if resetn = '0' then
            nb_cycle <= (others => '0');
            led_drive <= '0';
        elsif(rising_edge(clk)) then
            led_drive <= d_led_drive;
            nb_cycle <= d_cycle;
        end if ;
    end process;

    -- pour d_cycle : cpt_raz prioritaire
    -- pas d'increment si cpt_hold = '1'
    -- increment si cpt_count = '1' sur le niveau a '1' de end_counter et q_led_drive
    d_cycle <= (others => '0') when cpt_raz = '1' else
        nb_cycle + 1 when cpt_count = '1' and end_counter = '1' and cpt_hold = '0' and led_drive = '1' else
        nb_cycle ;
    -- driver de LED qui alterne à chaque fin de comptage end_counter
    d_led_drive <= not led_drive when end_counter = '1' else led_drive;
    -- buffers sur les sorties
    out_nb_cycle <= nb_cycle;
    out_led_drive <= led_drive;
end architecture;

```

Appel du compteur unitaire
compteur_unit

Process séquentiels de contrôle des registres
cycle et led_drive

Process combinatoire
Calcul des cycles et du contrôle des LED

4. Tester votre architecture avec un test bench.

- Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn, cpt_raz, cpt_hold, cpt_count

Liste des signaux de sortie à vérifier : nb_cycle, led_drive

Liste des signaux internes à vérifier : end_counter, d_cycle, d_led_drive, compteur_unit : q_counter

L'horloge clk est définie à 100Mhz

Note : un cycle de comptage est la durée de 2 comptages du counter_unit

=2* cible * period (clk)

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 200ns

- Déroulé de la validation et du du test bench

1. Test du resetn au démarrage

1.1. Les entrées seront définies comme suit :

- resetn = 0
- cpt_raz = 0 cpt_hold = 0 cpt_count = 0

1.2. Les sorties seront vérifiées

- nb_cycle = 0 : registre du compteur de cycle resetté
- led_drive = 0 : la LED est éteinte: registre du driver de LED resetté

1.3. Les signaux internes à vérifier

- d_cycle = 0

- `end_counter = 0` : counter_unit resetté
 - `compteur_unit : q_counter = 0` : counter_unit resetté
- 1.4. Sur le test bench
- on teste `nb_cycle` et `led_drive` après $1 \times \text{periode}(\text{clk}) + 1\text{ns}$
 - on attend $3 \times \text{period} + 1\text{ns}$ pour lancer la suite
- 2. Levée du reset**
- 2.1. Les entrées sont modifiées comme suit
- **`resetn = 1`**
 - `cpt_raz = 0` `cpt_hold = 0` `cpt_count = 0`
- 2.2. Les sorties seront vérifiées
- `nb_cycle = 0` : registre du compteur de cycle resetté
 - `led_drive = 0/1` à chaque fin de comptage du compteur_unit (la LED est allumée pendant $\text{cycle} \times \text{period}(\text{clk})$ secondes)
- 2.3. Les signaux internes à vérifier
- `d_cycle = 0`
 - **compteur_unit : q_counter** s'incrémente sur les front d'horloge et revient à 0 quand il atteint la valeur de cible-1
 - **end_counter** : passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
- 2.4. Sur le test bench
- on teste `nb_cycle = 0` et `led_drive = 1` après $(2 \times (2 \times \text{cible}) + \text{cible}) \times \text{periode}(\text{clk}) - 1\text{ns}$. On attend $2 \times (2 \times \text{cible}) \times \text{periode}(\text{clk})$ pour bien vérifier que `nb_cycle` reste à 0. Puis on attend $\text{cible} \times \text{period}(\text{clk})$ en plus pour tester si la LED allumée
 - on attend $3 \times (2 \times \text{cible}) \times \text{period} + 2\text{ns}$ pour lancer la suite.
 - Note on ajoute quelques ns sur les test et les changements sur les entrées pour vérifier les évènements synchrone ou asynchrone.
- 3. Autorisation de comptage du nombre de cycle**
- 3.1. Les entrées sont modifiées comme suit
- **`cpt_count = 1`**
 - `cpt_raz = 0` `cpt_hold = 0` `resetn = 1`
- 3.2. Les sorties seront vérifiées
- **nb_cycle** s'incrémente tous les $2 \times \text{cycle} \times \text{period}(\text{clk})$ secondes. Comme `nb_cycle` est défini sur 2 bits , le comptage donne un cycle 0 1 2 3 0 1 2 ...
 - `led_drive` alterne entre 0 et 1 toutes les $\text{cycle} \times \text{period}(\text{clk})$ secondes
- 3.3. Les signaux internes à vérifier
- **d_cycle**: fonctionne comme `nb_cycle`. mais change d'état un coup de clk plus tôt que `nb_cycle`
 - `compteur_unit : q_counter` s'incrémente sur les front d'horloge et revient à 0 quand il atteint la valeur de cible-1
 - `end_counter` : passe à 1 sur une période de clk quand le compteur_unit compte cible -1
- 3.4. Sur le test bench
- on teste `nb_cycle = "11"` et `led_drive = 1` après $(3 \times 2 \times \text{cible} + \text{cible}) \times \text{periode}(\text{clk}) + 1\text{ns}$. On attend $3 \times (2 \times \text{cible}) \times \text{periode}(\text{clk})$ pour compter `nb_cycle` jusqu' à 3 . Puis on attend $\text{cible} \times \text{period}(\text{clk})$ en plus pour tester si la LED allumée
 - on attend $4 \times \text{cible} \times \text{period} + 30\text{ns}$ pour lancer la suite afin d'être sur un état du compteur de cycle différent de 0 avec une LED allumée
- 4. Remise à zéro du compteur de cycle lors d'un comptage**
- 4.1. Les entrées sont modifiées comme suit
- **`cpt_raz = 1`**
 - `cpt_count = 1` `cpt_hold = 0` `resetn = 1`
- 4.2. Les sorties seront vérifiées
- **nb_cycle = "00"** au prochain front montant de l'horloge puis reste à 0
 - `led_drive` continue à alterner entre 0 et 1 toutes les $\text{cycle} \times \text{period}(\text{clk})$ secondes
- 4.3. Les signaux internes à vérifier
- **d_cycle** passe à 0 dès que `cpt_raz='1'` de façon asynchrone

- compteur_unit : q_counter s'incrémente sur les front d'horloge et revient à 0 quand il atteint la valeur de cible-1
 - end_counter : passe à 1 sur une période de clk quand le compteur_unit compte cible -1
- 4.4. Sur le test bench
- on teste nb_cycle = "00" après $1 \cdot \text{periode}(\text{clk}) + 1\text{ns}$ pour être sur d'avoir eu la synchronisation
 - on attend $2 \cdot 2 \cdot \text{cible} \cdot \text{period} + 1\text{ns}$ pour lancer la suite
- 5. Reprise du comptage après remise à zéro**
- 5.1. Les entrées sont modifiées comme suit
- **cpt_raz = 0**
 - cpt_count = 1 cpt_hold = 0 resetn = 1
- 5.2. Les sorties seront vérifiées
- **nb_cycle** s'incrémente de nouveau en recommençant de zéro
 - led_drive alterne entre 0 et 1 toutes les $\text{cycle} \cdot \text{periode}(\text{clk})$ secondes
- 5.3. Les signaux internes à vérifier
- **d_cycle** repart de zéro pour le comptage de cycle
 - compteur_unit : q_counter s'incrémente sur les front d'horloge et revient à 0 quand il atteint la valeur de cible-1
 - end_counter : passe à 1 sur une période de clk quand le compteur_unit compte cible -1
- 5.4. A noter
- Le premier cycle après le redémarrage n'est pas complet car l'alternance suit le comptage de compteur_unit qui n'est pas remis à zéro.
 - Si l'évènement cpt_raz = 0 intervient à l'instant t pour lequel counter_unit: q_counter = x ≠ 0 alors le premier cycle (nb_cycle = "00" durera (cible -x) * periode(clk)
- 5.5. Sur le test bench
- on teste nb_cycle = "10" après $((2 \cdot 2 \cdot \text{cible}) \cdot \text{periode}(\text{clk}) + 1\text{ns})$. On attend $2 \cdot (2 \cdot \text{cible}) \cdot \text{periode}(\text{clk})$ pour compter nb_cycle jusqu' à 2 .
 - on attend $2 \cdot \text{cible} \cdot \text{period} + 1\text{ns}$ pour lancer la suite
- 6. Maintien de la valeur du compteur de cycle lors d'un comptage**
- 6.1. Les entrées sont modifiées comme suit
- **cpt_hold = 1**
 - cpt_count = 1 cpt_raz = 0 resetn = 1
- 6.2. Les sorties seront vérifiées
- **nb_cycle** valeur maintenue tant que ctp_hold =1
 - led_drive continue à alterner entre 0 et 1 toutes les $\text{cycle} \cdot \text{periode}(\text{clk})$ secondes
- 6.3. Les signaux internes à vérifier
- **d_cycle** maintient aussi sa valeur
 - compteur_unit : q_counter s'incrémente sur les front d'horloge et revient à 0 quand il atteint la valeur de cible-1
 - end_counter : passe à 1 sur une période de clk quand le compteur_unit compte cible -1
- 6.4. Sur le test bench
- on teste nb_cycle = "10" et led_drive = 1 après $1 \cdot 2 \cdot \text{cible} \cdot \text{periode}(\text{clk}) + 1\text{ns}$. On attend 1 période de comptage de cycle pour s'assurer que le compteur de cycle est stoppé
 - on attend $2 \cdot 2 \cdot \text{cible} \cdot \text{period}(\text{clk}) + 2\text{ns}$ pour lancer la suite
- 7. Reprise du comptage après un maintien**
- 7.1. Les entrées sont modifiées comme suit
- **cpt_hold = 0**
 - cpt_count = 1 cpt_raz = 0 resetn = 1
- 7.2. Les sorties seront vérifiées
- **nb_cycle** s'incrémente de nouveau en recommençant de la valeur au moment du maintien
 - led_drive alterne entre 0 et 1 toutes les $\text{cycle} \cdot \text{periode}(\text{clk})$ secondes
- 7.3. Les signaux internes à vérifier
- **d_cycle** s'incrémente de nouveau en recommençant de la valeur au moment du maintien

- compteur_unit : q_counter s'incrémente sur les front d'horloge et revient à 0 quand il atteint la valeur de cible-1
- end_counter : passe à 1 sur une période de clk quand le compteur_unit compte cible -1

7.4. A noter

- Le premier cycle après le redémarrage n'est pas complet car l'alternance suit le comptage de compteur_unit qui n'est pas remis à zéro.
- Si l'évènement cpt_hold = 0 intervient à l'instant t pour lequel counter_unit: q_counter = x ≠ 0 alors le premier cycle (nb_cycle = "00" durera (cible -x) * periode(clk)

7.5. Sur le test bench

- on teste nb_cycle = "01" après $2 \times 2 \times \text{cible} \times \text{periode}(\text{clk}) + 1\text{ns}$. On attend $2 \times (2 \times \text{cible}) \times \text{periode}(\text{clk})$ pour compter nb_cycle jusqu'à 1 en partant de 3.
- on attend $1 \times \text{cible} \times \text{periode} + 1\text{ns}$ pour lancer la suite

8. test des priorité des commandes : cpt_raz est prioritaire sur ctp_hold et cpt_count

8.1. Les entrées sont modifiées comme suit

- départ : cpt_count = 1 cpt_hold = 0 cpt_raz = 0 resetn = 1
- puis : cpt_count = 1 cpt_hold = 1 cpt_raz = 0 resetn = 1
- puis : cpt_count = 1 cpt_hold = 1 cpt_raz = 1 resetn = 1
- puis : cpt_count = 1 cpt_hold = 1 cpt_raz = 0 resetn = 1
- puis : cpt_count = 1 cpt_hold = 0 cpt_raz = 1 resetn = 1
- changement toutes de $s \ 2 \times 2 \times \text{cible} \times \text{periode}(\text{clk})$ pour laisser 2 comptages de cycle entre chaque modifications.

8.2. Les sorties seront vérifiées

- **nb_cycle** va s'incrémenter puis se maintenir puis de remettre à zéro puis se maintenir puis de remettre à compter
- led_drive alterne entre 0 et 1 toutes les cycle*periode(clk) secondes

8.3. Sur le test bench

- on teste nb_cycle toutes les $2 \times 2 \times \text{periode}(\text{clk}) + 1\text{ns}$. et on vérifie les remise à 0 si les trois entrées sont à 1

• Test bench

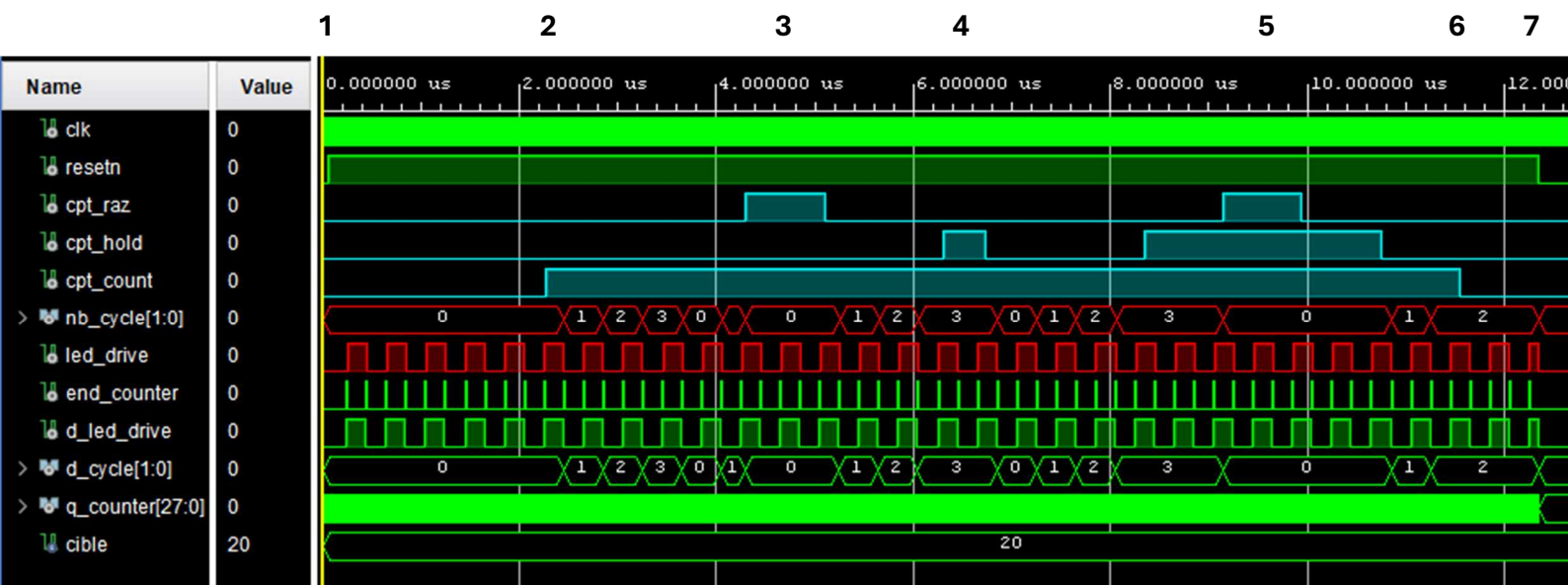
Le fichier est très long je ne l'ai pas mis dans le document tb_counter_led.vhd

- Résultat des test automatiques du test_bench

```
# run 14000ns
Note: PASS : TEST RESET (resetn = 0) >> nb_cycle =00 and led_drive = 0 OK
Time: 31 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : TEST LEVEE RESET (resetn = 1) >> nb_cycle =00 and led_drive = 1 après 2.5 cycle de comptage OK
Time: 1061 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : Comptage jusqu'a 3 >> nb_cycle =11 and led_drive = 1 OK
Time: 3463 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : TEST RAZ (cpt_raz = 1) >> nb_cycle =00 OK
Time: 4304 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : Comptage jusqu'a 2 après RAZ >> nb_cycle =10 OK
Time: 5906 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : TEST HOLD >> nb_cycle =11 OK
Time: 6708 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : Comptage jusqu'a 1 après HOLD
Time: 7541 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : Comptage bloqué a 3
Time: 9143 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : TEST priorité RAZ sur HOLD >> nb_cycle =00
Time: 9944 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : HOLD maintenue après RAZ
Time: 10745 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : reprise Comptage jusqu'a 2 après HOLD et RAZ
Time: 11546 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : arrêt du Comptage à 2 , le 2 est maintenu
Time: 12347 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: Nombre de test FAIL : 0
Time: 12347 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: PASS : test du resetn
Time: 13148 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 13148 ns Iteration: 0 Process: /tb_counter_led/line_59 File: D:/Vivado/fsm/fsm.srcs/sim_1/new/tb_counter_led.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_counter_led_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 14000ns
) launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:07 . Memory (MB): peak = 1230.844 ; gain = 0.000
```

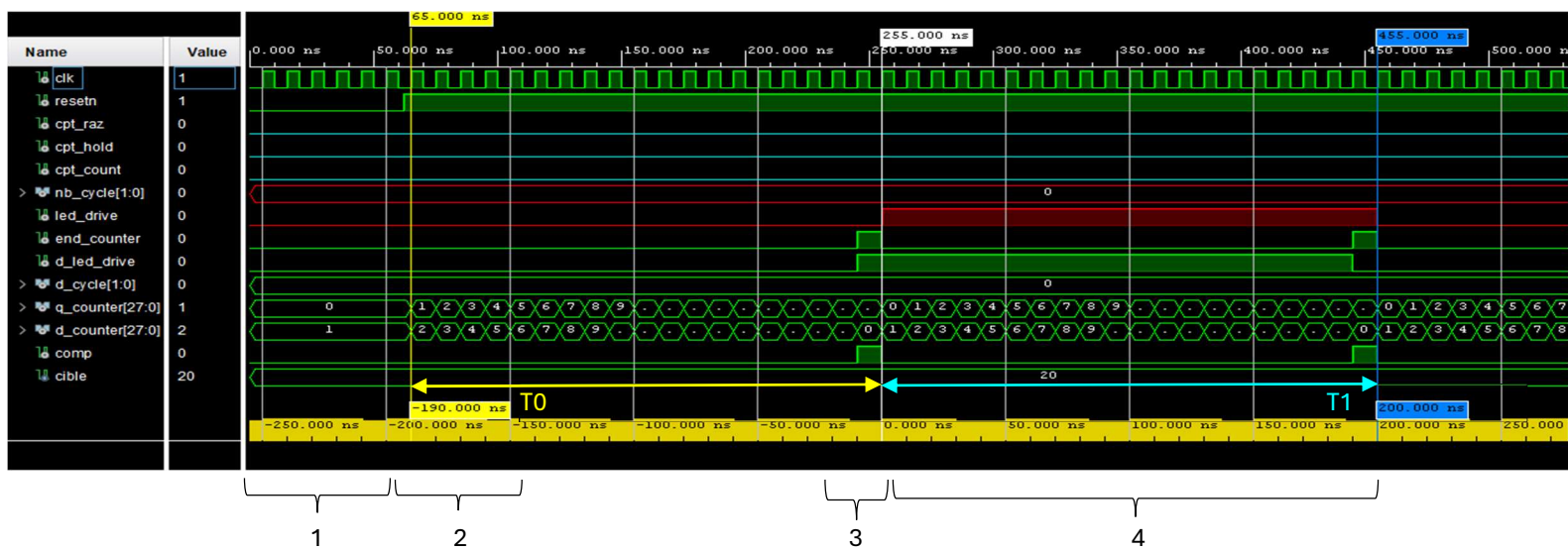
Les test automatiques ont été réalisés comme sur la description du paragraphe précédent . Tout est PASS.

- Validation fonctionnelle du VHDL



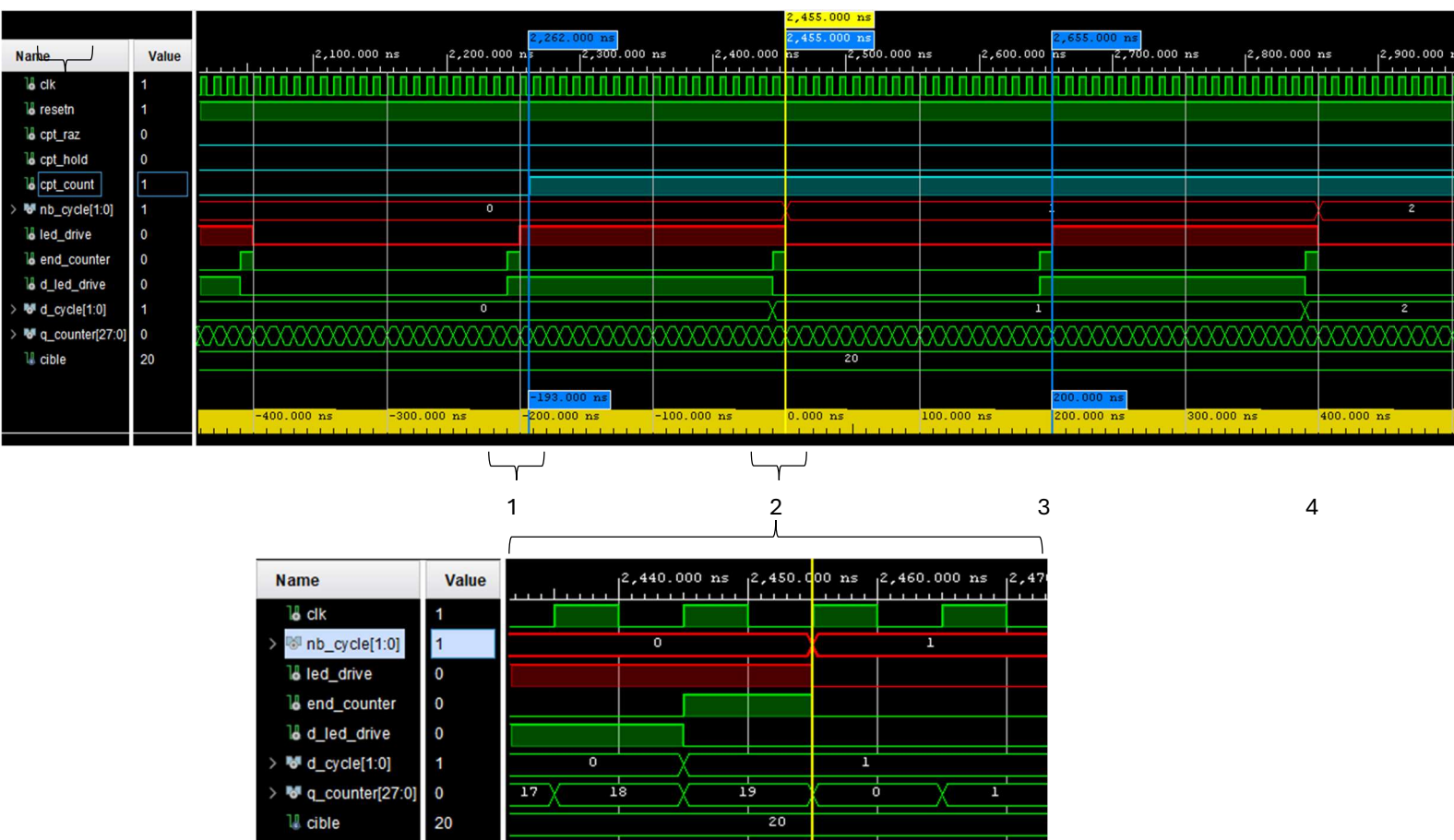
- 1- Levée du reset du système resetn = '1': le compteur_unit démarre (q_counter compte); led_drive commence à alterner 0 et 1
- 2- Autorisation de comptage cpt_count= 1 : nb_cycle s'incrémente.
- 3- Mise à zéro du compteur de cycle : cpt_raz = 1 => nb_cycle = 0 . Puis cpt_raz = 0 , reprise du comptage à partir de zéro OK
- 4- Hold du compteur : cpt_hold = 1 => nb_cycle figé. Puis cpt_hold = 0 , reprise du comptage de cycle là où il en était
- 5- Test des priorités: cpt_raz remet à zéro le compteur de cycle quelque soit les états de cpt_hold et cpt_count
- 6- Fin de comptage cpt_count = 0 => la sortie nb_cycle reste figée
- 7- Reset du système avec resetn : nb_cycle et led_drive repassent à 0

- Validation Démarrage après reset



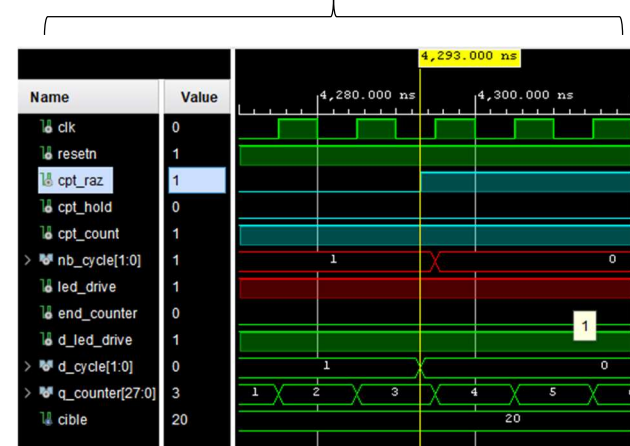
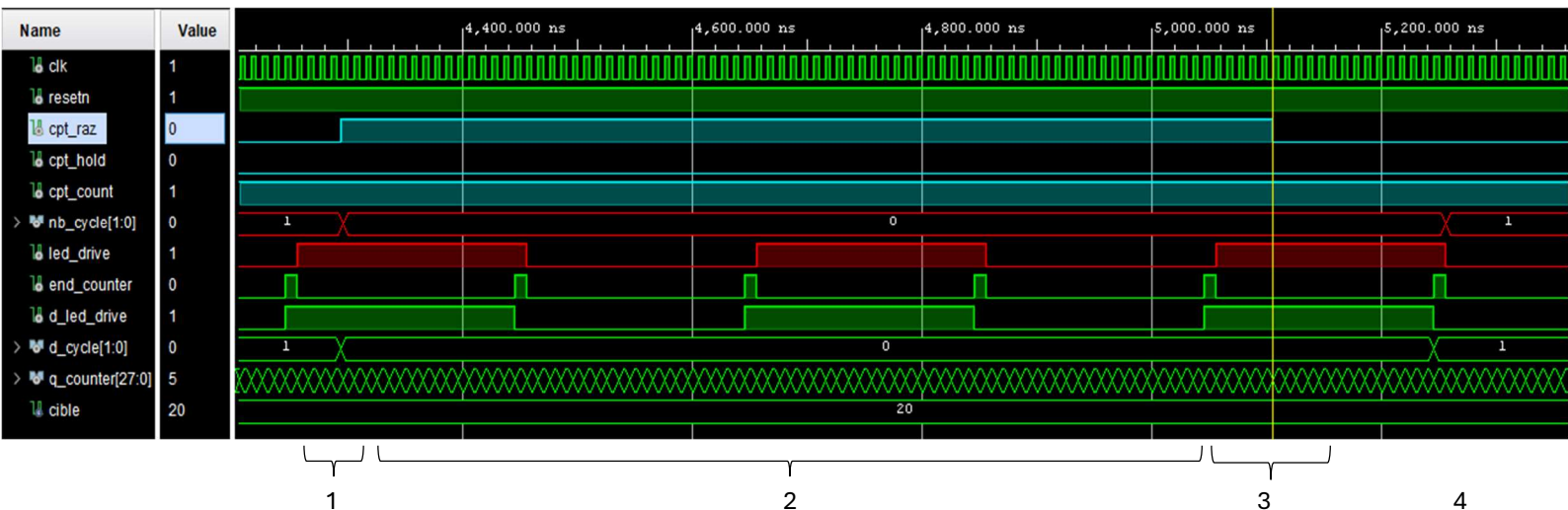
- 1-Quand resetn est à 0: tous les registres sont resettés led_drive, nb_cycle , counter_unit: q_counter
- 2-Quand resetn passe à 1 : le compteur unitaire s'incrémente sur les fronts montants d'horloge, le premier cycle de comptage n'est pas complet car au redémarrage d_counter = 1 en raison de la logique combinatoire de d_counter donc q_counter reçoit 1 au lieu de zéro sur le premier front d'horloge.
le premier cycle de comptage dure donc T0 = 190ns au lieu de T1 = 200ns comme pour le deuxième cycle
- 3-led_drive passe à 1 au front d'horloge suivant le niveau 1 de end_counter qui permet d'inverser led_drive
- 4- la LED reste on pendant T1= 200ns

- Validation Démarrage du compteur de cycle avec cpt_count



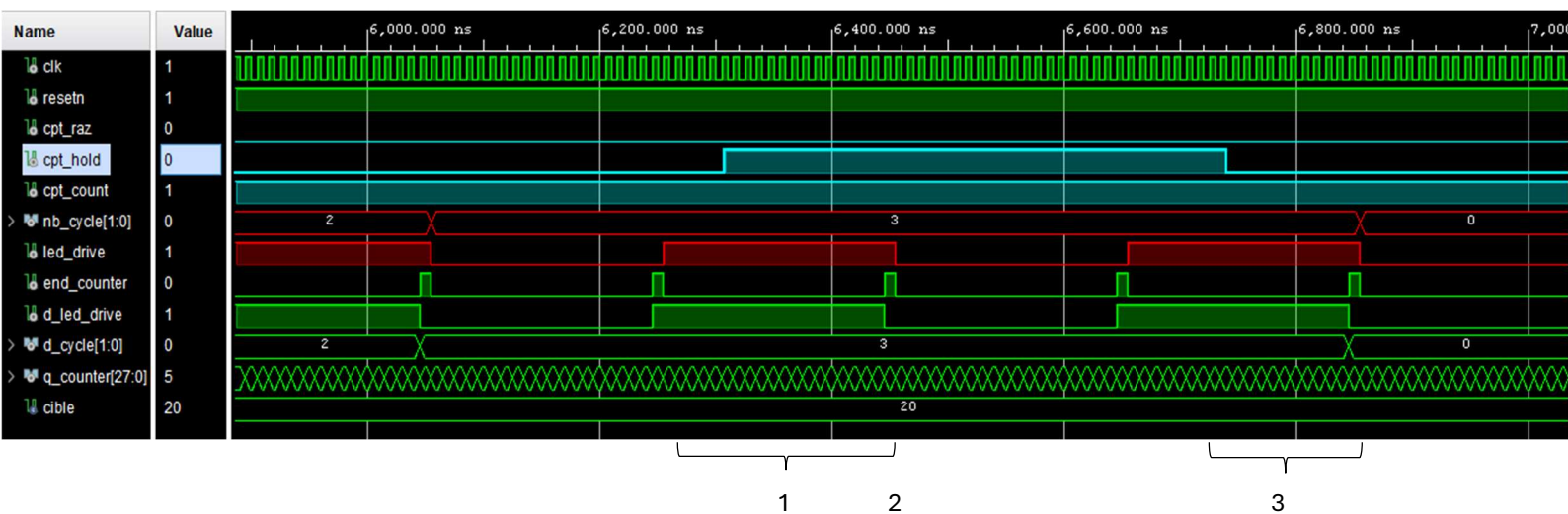
- 1-A la mise à 1 de cpt_count, le compteur de cycle est prêt à compter
- 2- nb_cycle s'incrémente au coup d'horloge suivant la fin d'un comptage unitaire quand end_counter et led_drive basculent
- 3-La durée d'un cycle est de 400ns
- 4- nb_cycle s'incrémente de nouveau après 2 cycles de comptage unitaire

- Validation remise à zéro du compteur de cycle avec cpt_raz



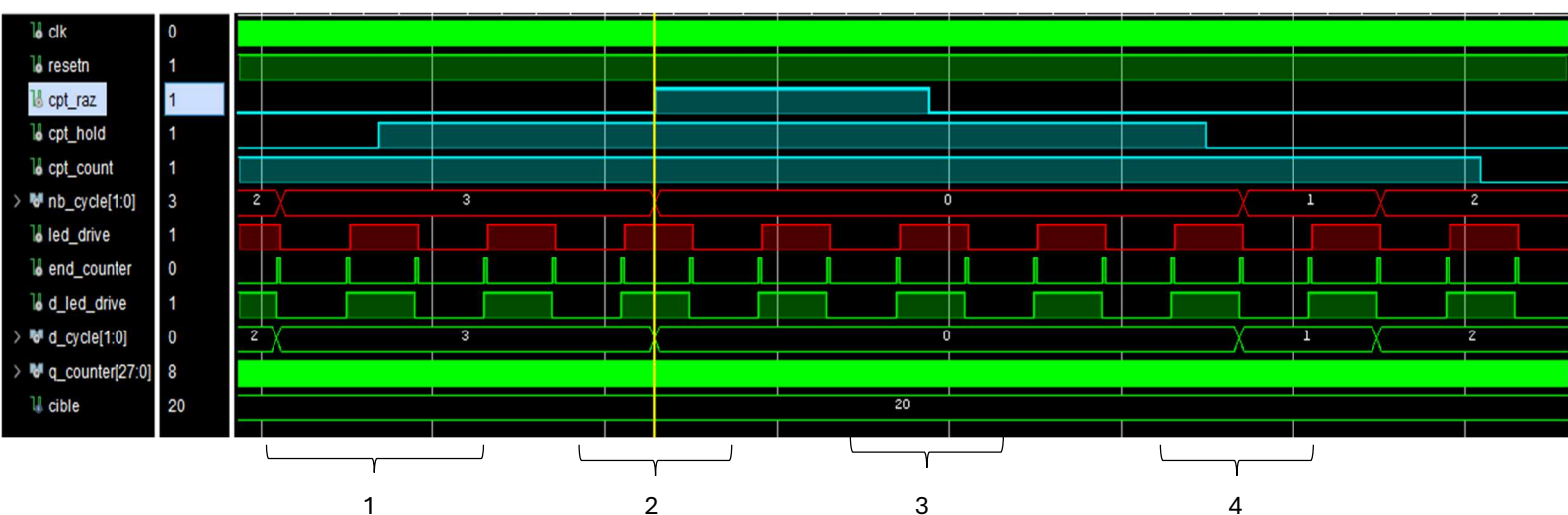
- 1- A la mise à zéro du compteur de cycle (cpt_raz = 1) : d_cycle passe à 0 immédiatement, le compteur de cycle nb_cycle est remis à 0 au coup d'horloge suivant
- 2- La sortie nb_cycle reste à 0 tant que RAZ = 0
La sortie led_drive continue d'alternier entre 0 et 1 a chaque fin de counter_unit
- 3- Quand on relâche la remise à zéro (cpt_raz = 0) le compteur de cycle recommence à s'incrémenter à la prochaine fin de comptage du compteur unitaire.
Comme pour le démarrage du comptage : la durée du premier cycle du compteur de cycle fonction de l'état du compteur unitaire quand les commandes d'entrées sont relâchées

- Validation du maintien du compteur de cycle avec cpt_hold



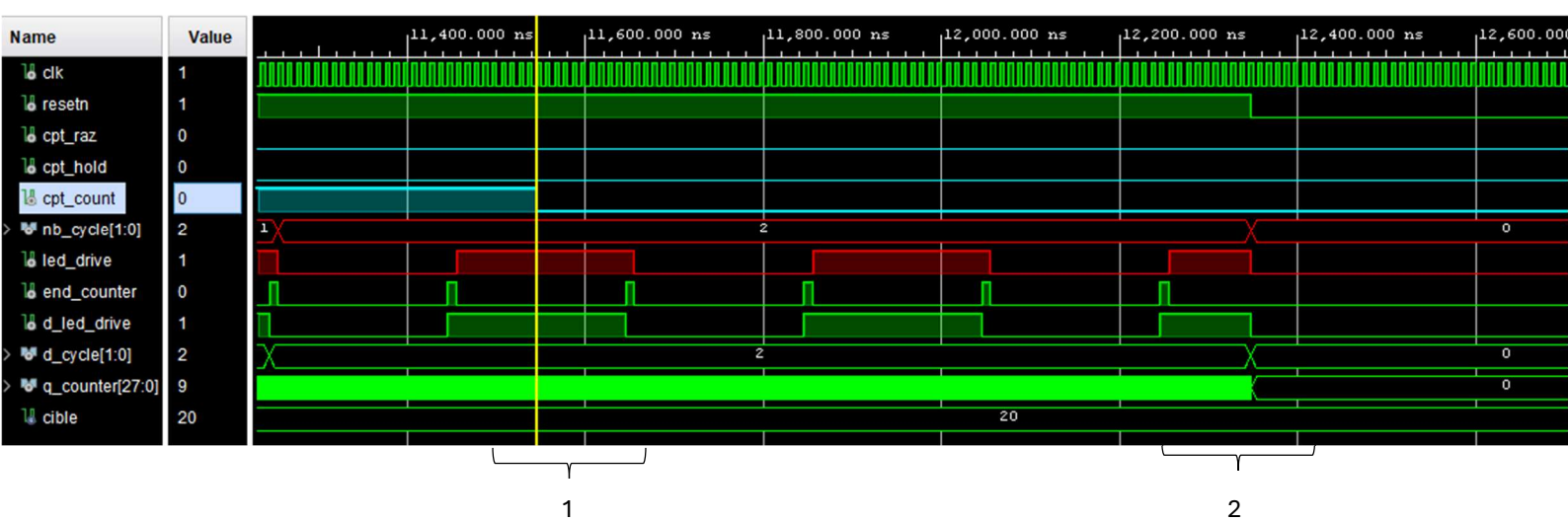
- 1- A l'activation du maintien (cpt_hold = 1), le compteur de cycle ne s'incrémente plus à la prochaine fin de comptage du compteur unitaire. La sortie led_drive continue d'alternier entre 0 et 1 a chaque fin de counter_unit
 - 2- nb_cycle reste à 3 à la fin de comptage du compteur unitaire.
 - 3- Quand on relâche le maintien (cpt_hold = 0) le compteur de cycle recommence à s'incrémenter à la prochaine fin de comptage du compteur unitaire.
- Comme pour le démarrage du comptage : la durée du premier cycle du compteur de cycle fonction de l'état du compteur unitaire quand les commandes d'entrées sont relâchées

- Validation des priorités entre count, hold et raz.



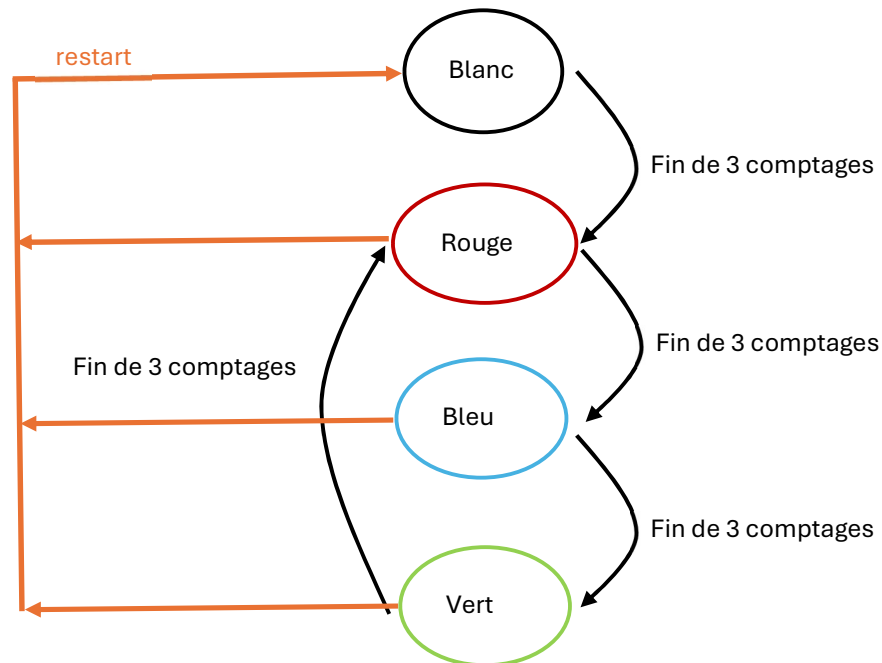
- 1- On lance un maintien (cpt_hold = 1) sur un comptage en cours la sortie nb_cycle est maintenue à 3
- 2- On lance un reset (cpt_raz = 1) pendant un comptage en maintien (cpt_count = 1 et cpt_hold = 1): la sortie nb_cycle est remise à 0 sur le front d'horloge suivant cpt_raz
- 3- Si on relâche cpt_raz, la sortie nb_cycle reste à zéro car le maintien est toujours actif
- 4- Le comptage de cycle redémarre quand cpt_hold passe à 0

- Validation de fin de comptage



- 1- Quand on autorise plus le comptage `cpt_count=0` le compteur de cycle ne s'incrémente plus, la sortie `nb_cycle` est maintenue
La sortie `led_drive` continue d'alternier entre 0 et 1 a chaque fin de `counter_unit`
- 2- un `resetn = 0` remet à zéro `nb_cycle` et arrête `led_drive`

5. Créez en RTL une machine à états (FSM) permettant de faire clignoter une LED RGB en rouge puis bleu et enfin en vert avant de recommencer le cycle (rouge, bleu, vert, ...). Dans chaque état la LED devra clignoter 3 fois. De plus, si le bouton restart est appuyé, on retourne dans l'état initial quel que soit l'état dans lequel on se situe. L'état initial est l'état dans lequel on se situe au démarrage, on passe à l'état rouge après 3 clignotements de la LED en blanc (rouge, vert et bleu actifs en même temps).

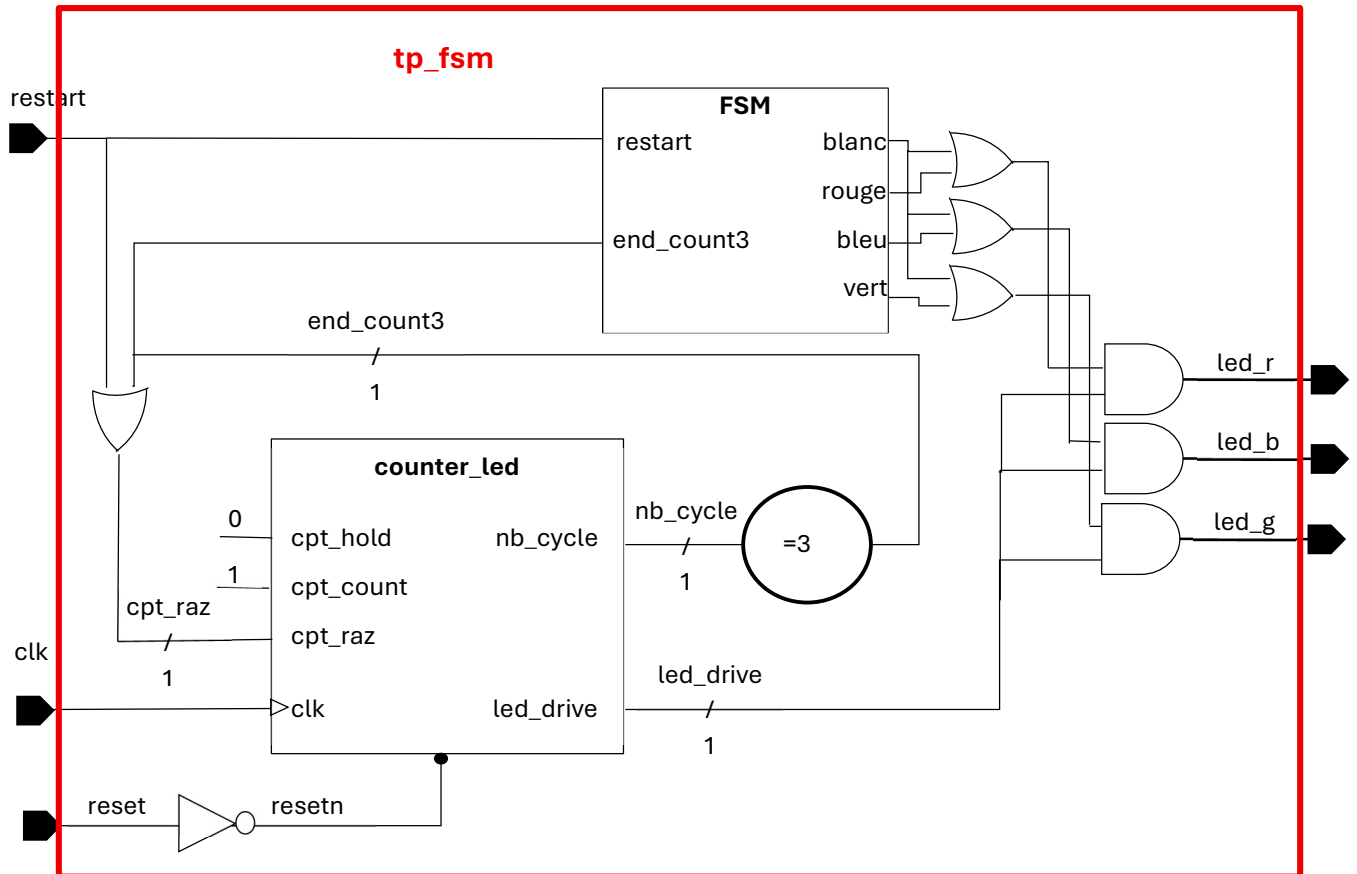


La machine à état comporte 4 états : blanc, rouge, bleu, vert

On passe d'un état à l'autre dans cet ordre blanc rouge->bleu->vert.

L'événement qui déclenche le passage d'un état à l'autre entre rouge->bleu->vert est un comptage de 3 cycles

Quelque soit l'état , on repasse à l'état blanc quand restart = 1



- on utilise le compteur de cycle créé précédemment pour compter jusqu'à 3 cycles et déclencher la machine d'état
- le counter_led est contrôlé par clk et resetn
 cpt_hold sera toujours à 0 car nous n'avons pas besoin de maintenir la sortie du compteur
 cpt_count sera toujours à 1 car le compteur doit compter tout le temps
 la remise à zéro (cpt_raz) se fera soit quand l'entrée restart ou à chaque comptage de 3 cycles soit
 $\text{cpt_raz} = \text{restart} \text{ or } \text{end_count3}$
- les signaux de led rouge, bleu et vert seront contrôlés par le signal led_drive afin de clignoter toutes les 2 secondes. mais aussi en fonction de la sortie de la machine d'état. selon la logique combinatoire suivante
 $\text{led_r} = \text{led_drive} \text{ and } (\text{rouge} \text{ or } \text{blanc})$
 $\text{led_b} = \text{led_drive} \text{ and } (\text{bleu} \text{ or } \text{blanc})$
 $\text{led_g} = \text{led_drive} \text{ and } (\text{vert} \text{ or } \text{blanc})$

6. Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

Signaux d'entrée : clk, reset, restart

Note : le signal de reset a été configuré actif haut sur le toplevel pour être en accord avec le fonctionnement des boutons sur la carte

Signaux de sortie : led_r, led_b, led_g

Signaux internes : end_count3, led_drive, cpt_raz, blanc, rouge, bleu, vert

7. Ajoutez à votre code VHDL les éléments que vous venez de créer.

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

entity tp_fsm is
    generic (
        cible : integer := 20000000
    );
    port (
        clk      : in std_logic;
        reset    : in std_logic;
        restart   : in std_logic;
        led_r    : out std_logic;
        led_b    : out std_logic;
        led_g    : out std_logic
    );
end tp_fsm;

architecture behavioral of tp_fsm is

    type state is (blanc, rouge, bleu, vert);
    signal current_state : state; --etat dans lequel on se trouve actuellement
    signal next_state : state; --etat dans lequel on passera au prochain coup d'horloge
    signal resetn : std_logic:= '0';
    signal end_count3 : std_logic:= '0'; -- detection de 3 cycles comptés
    signal cpt_raz : std_logic := '0'; -- remise à zéro du compteur de cycle
    signal nb_cycle : std_logic_vector (1 downto 0);
    signal led_drive : std_logic := '0';

begin
    )
        cpt: entity work.counter_led(behavioral)
            generic map (
                cible => cible
            )
            port map (
                clk => clk,
                resetn => resetn,
                cpt_raz => cpt_raz,
                cpt_hold => '0',
                cpt_count => '1',
                out_nb_cycle => nb_cycle,
                out_led_drive => led_drive
            );
    )

    process(clk, resetn)
    begin
    )
        if resetn = '0' then
            current_state <= blanc;
            --cpt_count <= '0';
        elsif rising_edge(clk) then
            --cpt_count <= '1';
        )
            if restart = '1' then
                current_state <= blanc;
            else
                current_state <= next_state;
        )
            end if;
        end if;
    )
    end process;

```

Appel du compteur de cycle
compteur_led

Process séquentiels de contrôle
de la machine d'état

```

process(current_state, end_count3)
begin
    next_state <= current_state;
    case current_state is
        when blanc =>
            if end_count3 = '1' then
                next_state <= rouge;
            end if;
        when rouge =>
            if end_count3 = '1' then
                next_state <= bleu;
            end if;
        when bleu =>
            if end_count3 = '1' then
                next_state <= vert;
            end if;
        when vert =>
            if end_count3 = '1' then
                next_state <= rouge;
            end if;
        when others =>
            next_state <= blanc;
    end case;
end process;

```

Process séquentiels
de fonctionnement
de la machine d'état

```

resetn <= not reset;
end_count3 <= '1' when to_integer(unsigned(nb_cycle)) = 3
    else '0';
cpt_raz <= '1' when end_count3 = '1' or restart = '1'
    else '0';
led_r <= led_drive when current_state = rouge or current_state = blanc
    else '0';
led_b <= led_drive when current_state = bleu or current_state = blanc
    else '0';
led_g <= led_drive when current_state = vert or current_state = blanc
    else '0';

```

Process combinatoire

end behavioral;

Calcul des 3 cycles , de la remise à zéro du
compteur de cycle et du contrôle des LED

8. Ecrivez un testbench pour tester votre architecture. Vérifiez à la simulation que vous obtenez le résultat attendu.

- Liste des signaux à vérifier

Liste des signaux d'entrée à tester : reset, restart

Liste des signaux de sortie à vérifier : led_r, led_b, led_g

Liste des signaux internes à vérifier : end_count3, nb_cycle, cpt_raz, led_drive, counter_led: end_counter

Listes des états à vérifier: current_state, next_state

L'horloge clk est définie à 100Mhz

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 200ns

- Déroulé de la validation et du test bench

1. Test du reset au démarrage

1.1. Les entrées seront définies comme suit :

- **reset = 1**
- restart = 0

1.2. Les sorties seront vérifiées

- led_r = 0 , led_b = 0 , led_g = 0

1.3. Les signaux : états internes à vérifier

- led_drive = 0 et nb_cycle = 0 : counter_led=0 resetté
- counter_led : end_counter = 0 : counter_unit resetté
- current_state = blanc : au reset l'état de départ est blanc

1.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0, led_b=0, led_g=0 après $(2 \times (2 \times \text{cible}) + \text{cible}) \times \text{periode}(\text{clk}) + 1\text{ns}$
- on attend cible*period + 1ns pour lancer la suite

2. Test du démarrage après reset

2.1. Les entrées seront définies comme suit :

- **reset = 0**
- restart = 0

2.2. Les sorties seront vérifiées

- led_r = 0/1 , led_b = 0/1 , led_g = 0/1 les sorties basculent à chaque fin cycle en suivant led_drive

2.3. Les signaux : états internes à vérifier

- led_drive = 0/1 basculent toutes les $2 \times \text{cible} \times \text{periode}(\text{clk})$ secondes
- nb_cycle = 0 s'incrémente toutes les $2 \times \text{cible} \times \text{periode}(\text{clk})$ secondes
- end_count3 = 1 sur une période de clk quand nb_cycle = 3
- cpt_raz = 1 quand end_count3 = 1
- current_state = blanc : au reset l'état de départ est blanc

2.4. Vérification des états

- on vérifie que tous les $3 \times 2 \times \text{cible} \times \text{periode}(\text{clk})$ secondes le current_state change de blanc->rouge->bleu->vert->rouge

2.5. Sur le test bench

Note: durant les test automatiques, on ne peut tester que les signaux top level. Je n'ai pas trouvé si on pouvait remonter des signaux de la hiérarchie sans créer des nouveaux port qui ne seraient utilisés que pour le debug/test.

- on attend cible*period + 1ns pour se trouver sur le cycle unitaire de la LED allumée
- on teste les 3 sorties de led : led_r = 0, led_b = 0 , led_g = 0 après chaque période de $(2 \times \text{cible}) \times \text{periode}(\text{clk}) + 1\text{ns}$
- on vérifie successivement les états en rebouclant le test 3 fois pour chaque état .
- on attend $2 \times \text{cible} \times \text{periode}(\text{clk})$ secondes

3. Test du restart

3.1. Les entrées seront définies comme suit :

- `reset = 1`
- **`restart = 1`**

3.2. Les sorties seront vérifiées

- `led_r = 0/1` , `led_b = 0/1` , `led_g = 0/1`: la LED va continuer à clignoter en blanc

3.3. Les signaux : états internes à vérifier

- `led_drive = 0/1` basculent toutes les $2 \cdot \text{cible} \cdot \text{periode}(\text{clk})$ secondes
- `nb_cycle = 0` car le `counter_led` a sa remise à 0 active
- `end_count3 = 0` tant que le restart est actif
- `cpt_raz = 1`
- `current_state = blanc` : au restart on repart l'état blanc
- `counter_led` : end_counter con tinue de compter

3.4. Sur le test bench

- on teste les 3 sorties de led : `led_r = 1` , `led_b = 1` , `led_g = 1` pendant 4 cycle de $(2 \cdot (2 \cdot \text{cible}) + \text{cible}) \cdot \text{periode}(\text{clk}) + 1\text{ns}$
- on attend $\text{cible} \cdot \text{period} + 1\text{ns}$ pour lancer la suite

4. Test du démarrage après restart

4.1. Les entrées seront définies comme suit :

- `reset = 1`
- **`restart = 0`**

4.2. Les sorties seront vérifiées

- `led_r = 0/1` , `led_b = 0/1` , `led_g = 0/1` les sorties basculent à chaque fin cycle en suivant `led_drive`

4.3. Les signaux : états internes à vérifier

- `led_drive = 0/1` basculent toutes les $2 \cdot \text{cible} \cdot \text{periode}(\text{clk})$ secondes
- `nb_cycle = 0` s'incrémente toutes les $2 \cdot \text{cible} \cdot \text{periode}(\text{clk})$ secondes
- `end_count3 = 1` sur une période de `clk` quand `nb_cycle = 3`
- `cpt_raz = 1` quand `end_count3 = 1`
- `current_state = blanc` : au reset l'état de départ est blanc

4.4. Vérification des états

- on vérifie que tous les $3 \cdot 2 \cdot \text{cible} \cdot \text{periode}(\text{clk})$ secondes le `current_state` change de blanc->rouge->bleu->vert->rouge

4.5. Sur le test bench

Note: durant les test automatiques, on ne peut tester que les signaux top level. Je n'ai pas trouvé si on pouvait remonter des signaux de la hiérarchie sans créer des nouveaux port qui ne seraient utilisés que pour le debug/test.

- on attend $\text{cible} \cdot \text{period} + 1\text{ns}$ pour se trouver sur le cycle unitaire de la LED allumée
- on teste les 3 sorties de led : `led_r = 0` , `led_b = 0` , `led_g = 0` après chaque période de $(2 \cdot \text{cible}) \cdot \text{periode}(\text{clk}) + 1\text{ns}$
- on vérifie successivement les états en rebouclant le test 3 fois pour chaque état .

- **Test bench**

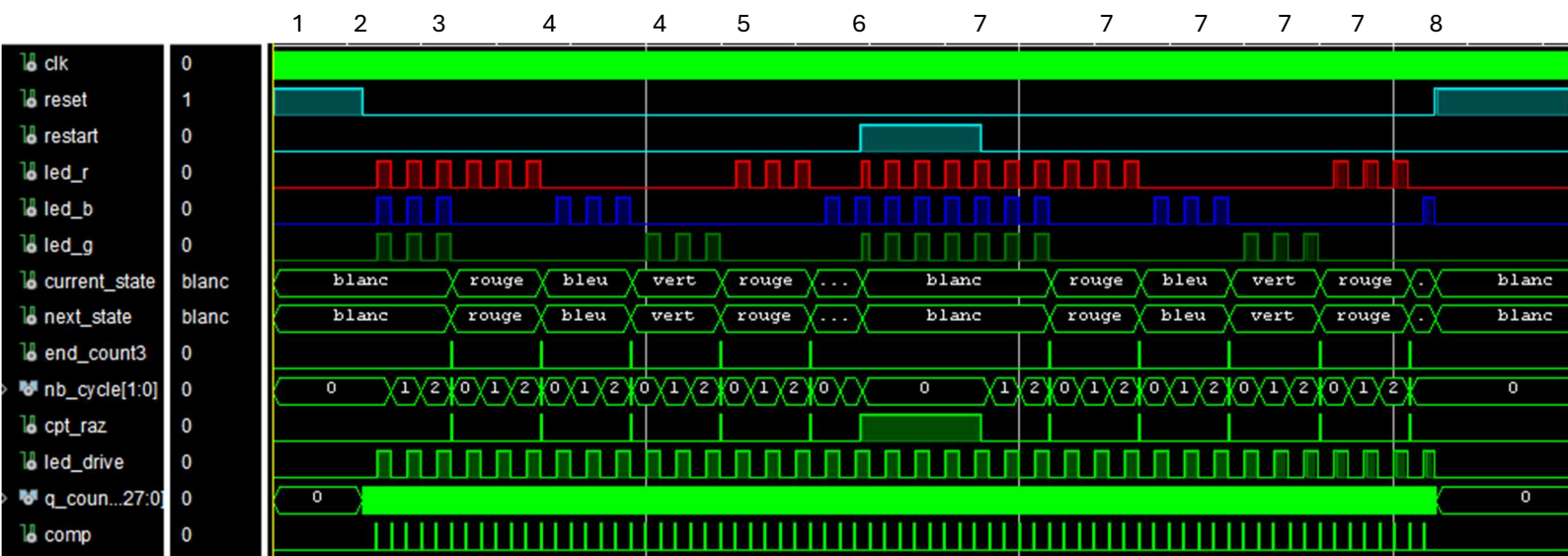
Le fichier est très long je ne l'ai pas mis dans le document : `tb_tp_fsm.vhd`

- Résultat des test automatiques du test_bench

```
# run 18000ns
Note: PASS : TEST reset : la led est eteinte OK
Time: 1001 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST démarrage après reset: led en blanc OK
Time: 1403 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST démarrage après reset: led en blanc OK
Time: 1804 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST démarrage après reset: led en blanc OK
Time: 2205 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 2606 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 3007 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 3408 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au bleu: led en bleu OK
Time: 3809 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au bleu: led en bleu OK
Time: 4210 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au bleu: led en bleu OK
Time: 4611 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au vert: led en vert OK
Time: 5012 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au vert: led en vert OK
Time: 5413 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au vert: led en vert OK
Time: 5814 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 6215 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 6616 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 7017 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST pendant restart: led en blanc OK
Time: 7889 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST pendant restart: led en blanc OK
Time: 8290 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST pendant restart: led en blanc OK
Time: 8691 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST pendant restart: led en blanc OK
Time: 9092 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST démarrage après restart: led en blanc OK
Time: 9504 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST démarrage après restart: led en blanc OK
Time: 9905 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST démarrage après restart: led en blanc OK
Time: 10306 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 10707 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 11108 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 11509 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au bleu: led en bleu OK
Time: 11910 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au bleu: led en bleu OK
Time: 12311 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au bleu: led en bleu OK
Time: 12712 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au vert: led en vert OK
Time: 13113 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au vert: led en vert OK
Time: 13514 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au vert: led en vert OK
Time: 13915 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 14316 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 14717 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: PASS : TEST passage au rouge: led en rouge OK
Time: 15118 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 15519 ns Iteration: 0 Process: /tb_tp_fsm/line_59 File: D:/Vivado/TP3_720_0/tb_tp_fsm.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_tp_fsm_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 18000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:07 . Memory (MB): peak = 1496.273 ; gain = 0.000
```

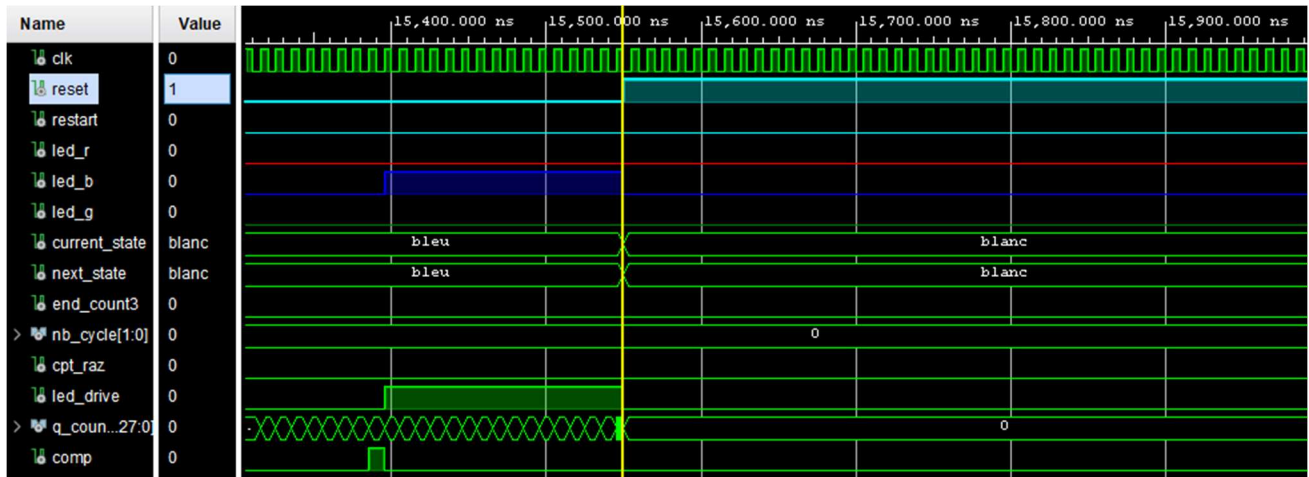
Tous les tests automatiques sont passés

- Validation fonctionnelle



1. Test du reset: la LED est éteinte, les compteurs sont remis à zéro (nb_cycle, q_counter), la FSM est en blanc
2. Démarrage après reset: la LED est allumée en blanc pendant 3 cycle de comptage
le compteur nb_cycle démarre pour compter jusqu' 2 (3 cycles)
led_drive alterne 0/1
3. Passage de la LED en rouge après 3 cycles blanc grâce à end_count3 = 1
nb_cycle est remis à "00" grâce à cpt_raz = 1
le current_state passe de blanc à rouge
4. Même fonctionnement que l'étape 4 pour le passage en bleu puis en vert
5. Passage de la LED en rouge après 3 cycles vert grâce à end_count3 = 1
nb_cycle est remis à 0 grâce à cpt_raz = 1
le current_state passe de rouge à vert
6. Test du restart : la LED passe en blanc
le current_state passe à blanc
le compteur nb_cycle est remis à "00" grâce à cpt_raz = 1
7. Redémarrage après restart
la FSM redémarre en blanc pour 3 cycles avant de repasser en rouge puis bleu puis vert puis rouge
le compteur de cycle est redémarré
8. Test du passage au reset: la LED est éteinte, les compteurs sont remis à zéro (nb_cycle, q_counter), la FSM est en blanc

- Validation du passage au reset



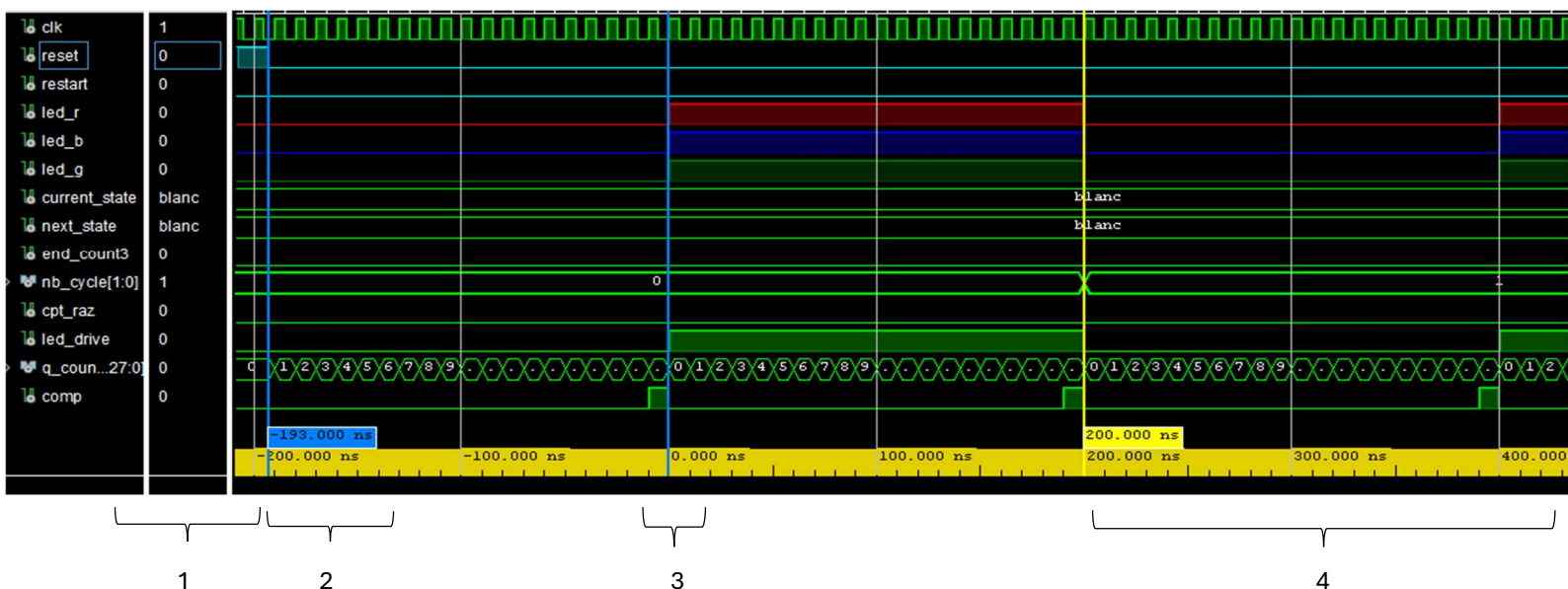
Au passage au reset: toutes les remises à 0 sont faites de façon asynchrone sur le signal reset

La LED est éteinte, le compteur de cycle et le compteur unitaire sont remis à 0

led_drive = 0

La machine d'état passe à blanc

- Validation du démarrage après reset



1- Quand reset est à 1: tous les registres sont resettés led_drive, nb_cycle. la fsm est dans l'état blanc

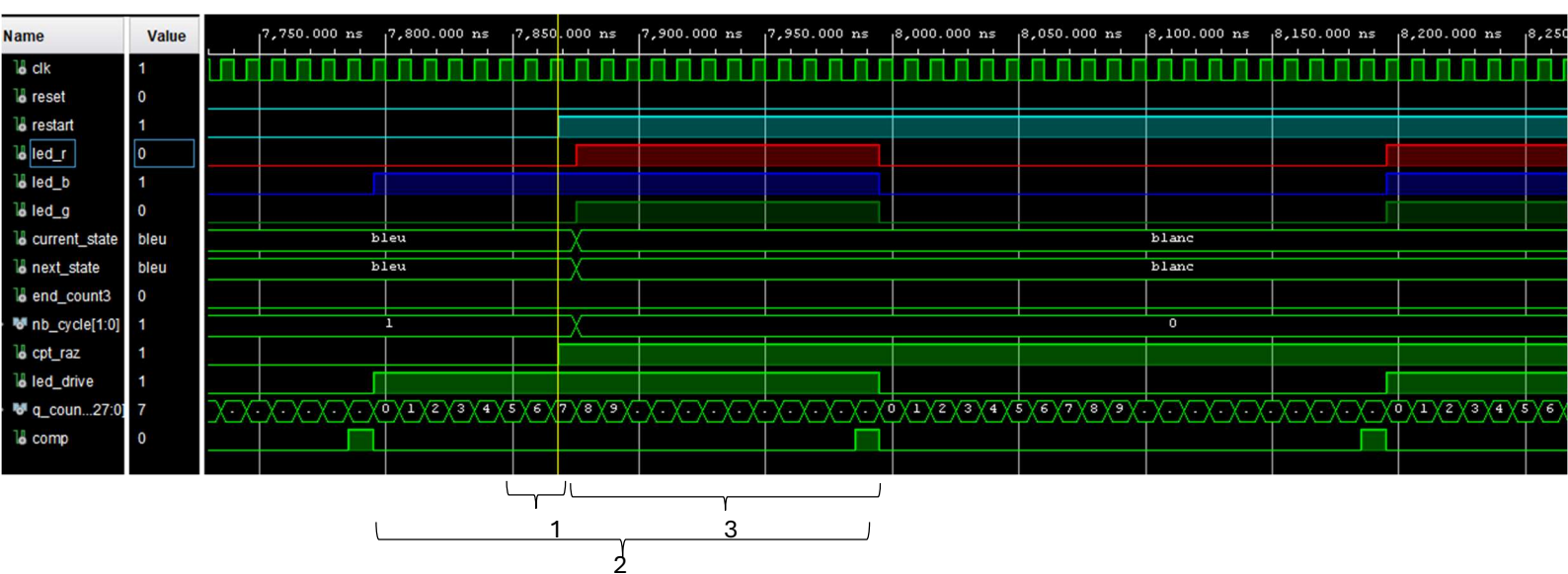
2- Quand reset passe à 0 : le compteur unitaire s'incrémente sur les fronts montants d'horloge, le premier cycle de comptage n'est pas complet car au redémarrage d_counter = 1 en raison de la logique combinatoire de d_counter donc q_counter reçoit 1 au lieu de zéro sur le premier front d'horloge.

le premier cycle de comptage dure donc $T_0 = 193\text{ns}$ au lieu de $T_1 = 200\text{ns}$ comme pour le deuxième cycle

3- la led s'allume en blanc après le premier cycle unitaire

4- le compteur de cycle d'incrément après 2 comptage de cycle unitaire

- Validation du restart



1-l'activation de restart (restart = 1) alors que la LED est en bleu

2- le signal led_drive n'est pas affecté par le restart : la LED reste allumée pendant la même période

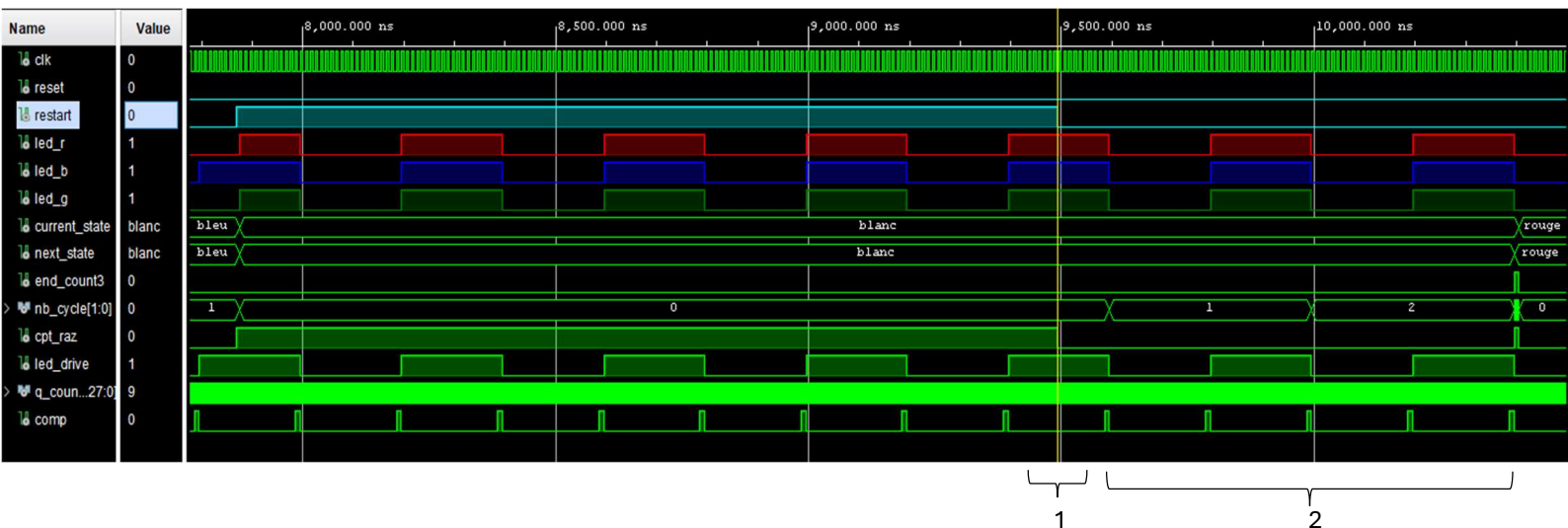
3-la remise à zéro de counter_led est activée de façon asynchrone en même temps de le restart

nb_cycle passe à 0 après un coup d'horloge

la machine d'état repart sur un état blanc sur le même coup d'horloge

la led change de couleur sur la même periode d'éclairage puis poursuit les 3 cycle de clignotement

- Validation de démarrage après restart

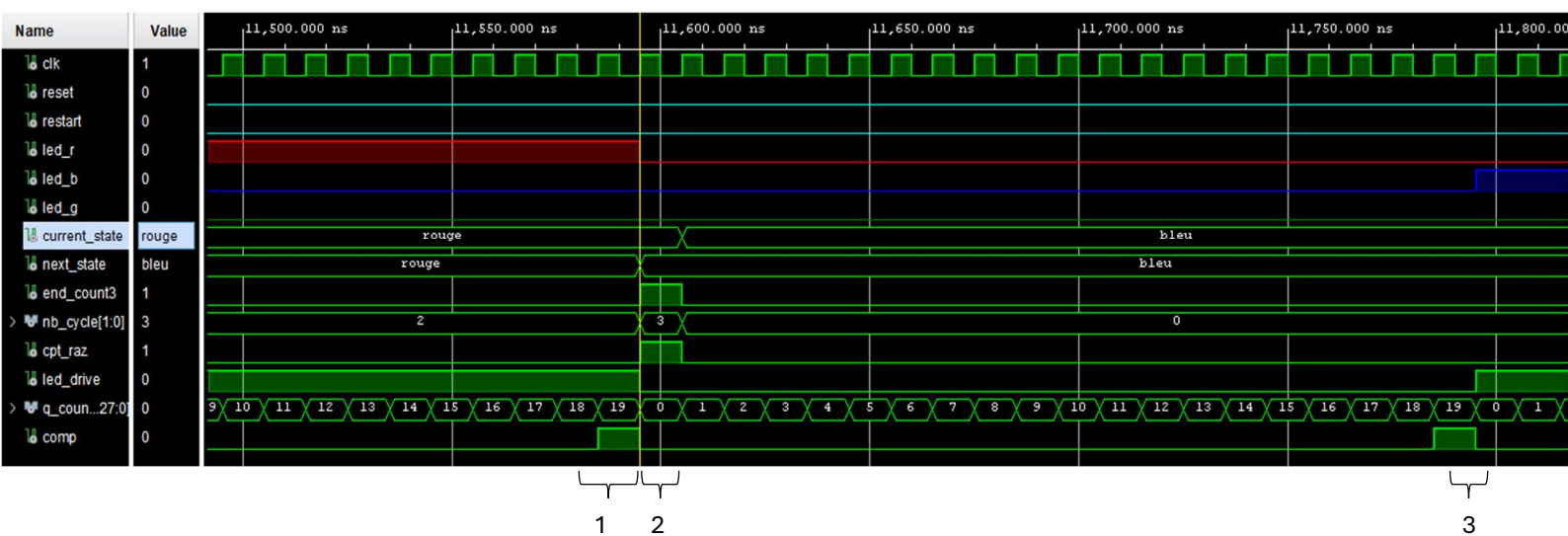


1-on relâche le restart (restart = 0), la remise à zéro du compteur de cycle est relâché nb_cycle s'incrémente

la LED était déjà en blanc pendant de restrtr , elle termine son cycle d'allumage en blanc

2-la fsm est en blanc sur 3 cycle et repasse en rouge quand nb_cycle passe a 3

- Validation du changement de couleur de la LED



- 1-quand le compteur unitaire arrive à cible-1 le signal led_drive passe à 0, la LEDs'eteint
le comptage des 3 cycles déclenche end_count3 =1 qui fait passer la FSM sur une autre couleur de façon synchrone
- 2- current_state change d'état au prochain coup d'horloge)
le compteur de cycle est remis à '00'
- 3- la LED a changé de couleur au cycle de comptage unitaire suivant

9. Exécutez la synthèse et relevez les ressources utilisées (y compris la FSM). Sur la schématique, identifiez où se situe votre compteur de cycle.

INFO: [Synth 8-802] inferred FSM for state register 'current_state_reg' in module 'tp_fsm'

State	New Encoding	Previous Encoding
blanc	0001	001
rouge	0010	010
bleu	0100	011
vert	1000	100

Les 4 états de la machine d'état ont bien été reconnus

INFO: [Synth 8-3354] encoded FSM with state register 'current_state_reg' using encoding 'one-hot' in module 'tp_fsm'

Finished RTL Optimization Phase 2 : Time (s): cpu = 00:00:12 ; elapsed = 00:00:14 . Memory (MB): peak = 1267.934 ; gain = 187.824

Start RTL Component Statistics

Detailed RTL Component Info :

```

+---Adders :
  2 Input  28 Bit    Adders := 1
  2 Input   2 Bit    Adders := 1
+---Registers :
      28 Bit    Registers := 1
      2 Bit     Registers := 1
      1 Bit     Registers := 1
+---Muxes :
  2 Input  28 Bit    Muxes := 1
  5 Input   4 Bit    Muxes := 1
  2 Input   2 Bit    Muxes := 1
  2 Input   1 Bit    Muxes := 3

```

Finished RTL Component Statistics

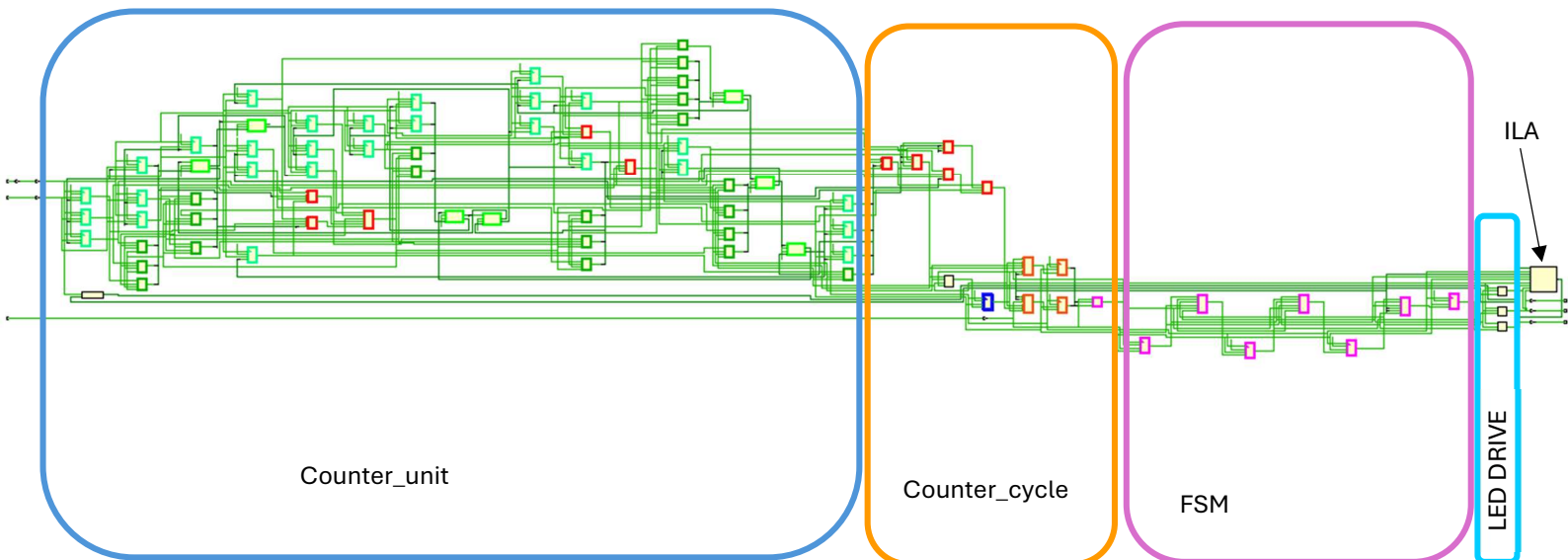
1 additionneur du counter_unit sur 28 bits
 1 additionneur du compteur de cycle sur 2 bits
 1 registre 28 bits pour la cible du counter_unit
 1 registre 2 bits pour la cible du compteur de cycle
 4 registres pour les états de la machine d'état
 1 mux 28 bits pour le counter_unit
 1 mux 4 bits/ 5 entrées pour les 5 états de la machine d'état
 1 mux 2 bits/ 2 entrées pour le compteur de cycle
 3 mux 1bit / 2 entrées pour les 3 leds

Report Cell Usage:

	Cell	Count
1	BUFG	1
2	CARRY4	7
3	LUT3	5
4	LUT4	28
5	LUT5	2
6	LUT6	6
7	FDCE	34
8	FDPE	1
9	IBUF	3
10	OBUF	3

- 1 BUFG : 1 buffer d'horloge
- 7 CARRY4 : 7 additionneurs de 4 bits pour le compteur 28 bits
- 5 LUT3
 - 3 LUT3 pour les 3 sorties de LED
 - 1 LUT3 pour le compteur 28 bits
 - 1 LUT3 pour la gestion de la machine d'état en fonction du nb_cycle
- 28 LUT4:
 - 20 LUT4 pour le compteur 28 bits
 - 7 LUT4 pour la logique sur le compteur de cycle
 - 1 LUT4 pour la logique sur le signal led_drive
- 2 LUT5: 5 LUT5 pour la logique sur le compteur de cycle
- 6 LUT6:
 - 3 LUT6 pour la gestion de la machine d'état
 - 3 LUT6 pour la logique sur le compteur de cycle
- 34 FCDE:
 - 28 registres pour le compteur
 - 1 registre pour le signal led_drive
 - 3 registres pour les 3 états (rouge, bleu, vert) de la machine d'etat
 - 2 registres pour le comptage du nombre de cycle
- 1 FDPE 1 registre pour l'état initial blanc de la machine d'état
- 3 IBUF 3 buffers d'entrée pour les 3 entrées
- 3 OBUF 3 buffers de sortie pour les 3 sorties

- Identification des blocs sur le schéma implémenté



10. Modifiez le fichier de contraintes pour connecter vos entrées / sorties du système avec les broches de la carte. Réglez l'horloge pour que sa fréquence soit à 100MHz.

```
# PL System Clock
set_property -dict { PACKAGE_PIN H16 IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L13P_T2_MRCC_35 Sch=sysclk
create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }];#set

# RGB LEDs
set_property -dict { PACKAGE_PIN L15 IOSTANDARD LVCMOS33 } [get_ports { led_b }]; #IO_L22N_T3_AD7N_35 Sch=led0_b
set_property -dict { PACKAGE_PIN G17 IOSTANDARD LVCMOS33 } [get_ports { led_g }]; #IO_L16P_T2_35 Sch=led0_g
set_property -dict { PACKAGE_PIN N15 IOSTANDARD LVCMOS33 } [get_ports { led_r }]; #IO_L21P_T3_DQS_AD14P_35 Sch=led0_r
#set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { led1_b }]; #IO_0_35 Sch=led1_b
#set_property -dict { PACKAGE_PIN L14 IOSTANDARD LVCMOS33 } [get_ports { led1_g }]; #IO_L22P_T3_AD7P_35 Sch=led1_g
#set_property -dict { PACKAGE_PIN M15 IOSTANDARD LVCMOS33 } [get_ports { led1_r }]; #IO_L23N_T3_35 Sch=led1_r

# Buttons
set_property -dict { PACKAGE_PIN D20 IOSTANDARD LVCMOS33 } [get_ports { reset }]; #IO_L4N_T0_35 Sch=btn[0]
set_property -dict { PACKAGE_PIN D19 IOSTANDARD LVCMOS33 } [get_ports { restart }]; #IO_L4P_T0_35 Sch=btn[1]
```

La fréquence de l'horloge sur la carte est de 125Mhz soit une période de 8ns.

Les signaux reset et restart sont reliés aux boutons pour le test.

11. Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique).

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 2,048 ns		Worst Hold Slack (WHS): 0,008 ns		Worst Pulse Width Slack (WPWS): 2,750 ns	
Total Negative Slack (TNS): 0,000 ns		Total Hold Slack (THS): 0,000 ns		Total Pulse Width Negative Slack (TPWS): 0,000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 4035		Total Number of Endpoints: 4019		Total Number of Endpoints: 2284	
All user specified timing constraints are met.					

Le design respecte les contraintes de timings pour une horloge à 125Mhz.

Il n'y a pas de violation de setup et de hold TNS = 0 et THS= 0.

From Clock: sys_clk_pin
To Clock: sys_clk_pin

Setup :	0	Failing Endpoints, Worst Slack	2.048ns,	Total Violation	0.000ns
Hold :	0	Failing Endpoints, Worst Slack	0.047ns,	Total Violation	0.000ns
PW :	0	Failing Endpoints, Worst Slack	2.750ns,	Total Violation	0.000ns

Max Delay Paths

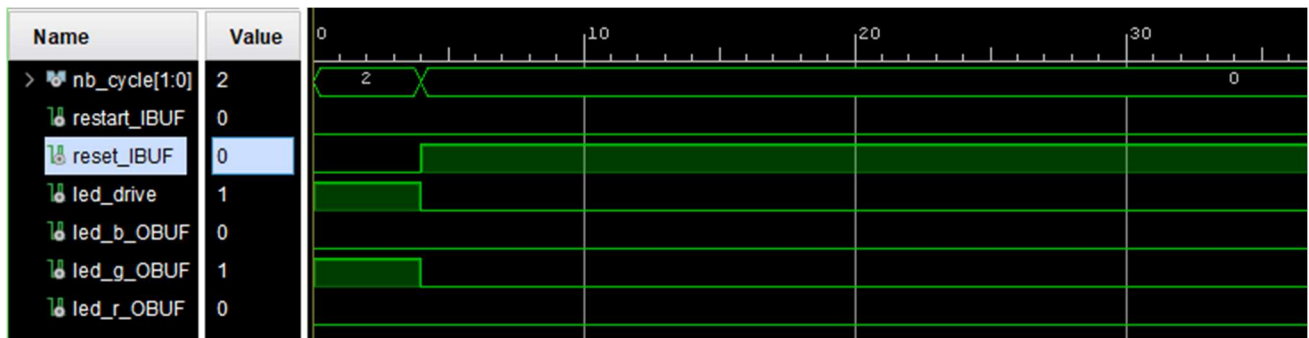
```
Slack (MET) : 2.048ns (required time - arrival time)
Source: <hidden>
          (rising edge-triggered cell FDRE clocked by sys_clk_pin (rise@0.000ns fall@4.000ns period=8.000ns))
Destination: <hidden>
          (rising edge-triggered cell FDRE clocked by sys_clk_pin (rise@0.000ns fall@4.000ns period=8.000ns))
Path Group: sys_clk_pin
Path Type: Setup (Max at Slow Process Corner)
Requirement: 8.000ns (sys_clk_pin rise@8.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay: 5.547ns (logic 1.242ns (22.391%) route 4.305ns (77.609%))
Logic Levels: 4 (LUT2=1 LUT5=1 LUT6=2)
Clock Path Skew: -0.165ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD): 4.843ns = ( 12.843 - 8.000 )
  Source Clock Delay (SCD): 5.298ns
  Clock Pessimism Removal (CPR): 0.290ns
Clock Uncertainty: 0.035ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
  Total System Jitter (TSJ): 0.071ns
  Total Input Jitter (TIJ): 0.000ns
  Discrete Jitter (DJ): 0.000ns
  Phase Error (PE): 0.000ns
```

Location	Delay type	Incr(ns)	Path(ns)	Netlist Resource(s)
(clock sys_clk_pin rise edge)				
		0.000	0.000 r	
H16		0.000	0.000 r	clk (IN)
	net (fo=0)	0.000	0.000	clk
H16	IBUF (Prop_ibuf_I_O)	1.457	1.457 r	clk_IBUF_inst/O
	net (fo=1, routed)	2.076	3.533	clk_IBUF
BUFGCTRL_X0Y16	BUFG (Prop_bufg_I_O)	0.101	3.634 r	clk_IBUF_BUFG_inst/O
	net (fo=1800, routed)	1.664	5.298	<hidden>
SLICE_X16Y29	FDRE		r	<hidden>
SLICE_X16Y29	FDRE (Prop_fdre_C_Q)	0.518	5.816 r	<hidden>
	net (fo=158, routed)	0.985	6.801	<hidden>
SLICE_X19Y25	LUT2 (Prop_lut2_I1_O)	0.150	6.951 f	<hidden>
	net (fo=8, routed)	1.041	7.992	<hidden>
SLICE_X20Y32	LUT6 (Prop_lut6_I1_O)	0.326	8.318 f	<hidden>
	net (fo=7, routed)	0.920	9.238	<hidden>
SLICE_X24Y20	LUT6 (Prop_lut6_I4_O)	0.124	9.362 r	<hidden>
	net (fo=11, routed)	0.676	10.038	<hidden>

Le chemin critique avec un slack de 2.048ns a été identifié dans l'ILA

12. Générez le bitstream pour vérifier le système sur carte.

- Validation du reset sur l'ILA



On trigge sur le signal reset_IBUF sur front montant

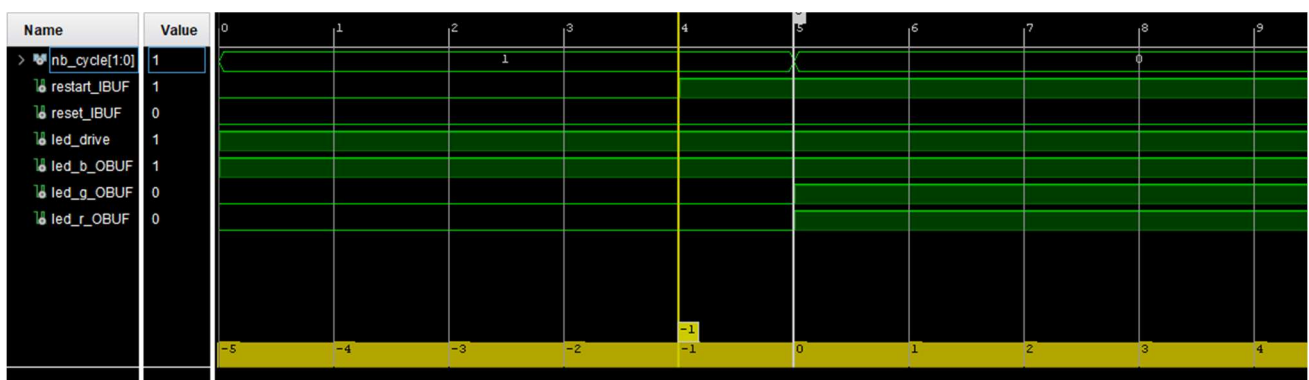
Le bouton reset (BTN0) est appuyé et maintenu sur le deuxième cycle d'allumage de la LED verte

=> la LED est éteinte immédiatement, tous les signaux de led passent à 0 (led_r = 0 led_b=0 et led_g = 0)

=> le compteur de cycle est immédiatement remis à 0 , nb_cycle=0

Note : le démarrage après reset peut être capturé sur un front descendant du reset mais les temps de premier comptage et d'allumage de la LED sont trop long pour être capturés sur la fenêtre de capture de l'ILA. La validation sera faire avec l'oscilloscope

- Validation du restart sur l'ILA



On trigge sur le signal restart_IBUF sur front montant

Le bouton restart (BTN1) est appuyé et maintenu sur le deuxième cycle d'allumage de la LED bleue

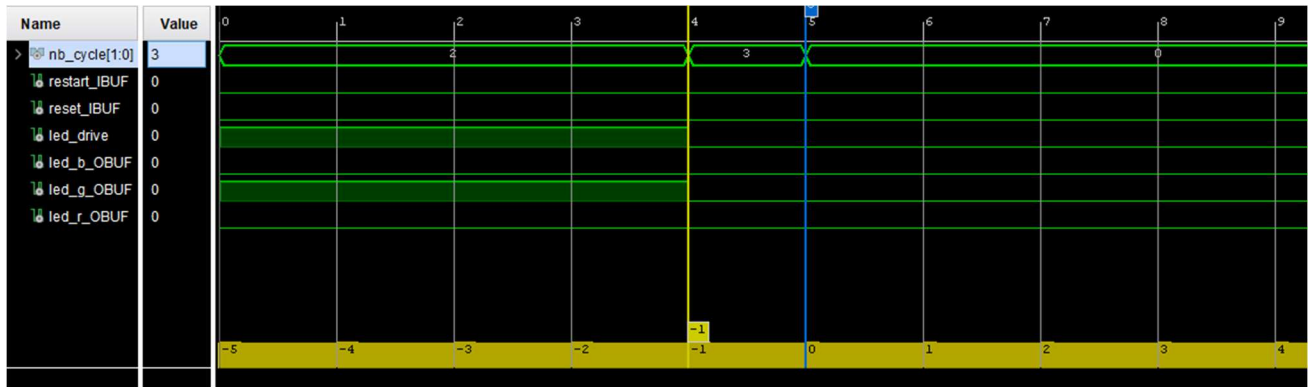
=> la LED s'allume en blanc au coup d'horloge suivant , tous les signaux de led passent à 1 (led_r = 1 led_b=1 et led_g = 1)

=> le compteur de cycle est remis à 0 au coup d'horloge suivant , nb_cycle=0

La mesure du délai entre les deux évènement donne unité de temps donc une unité d'échantillonnage soit une période d'horloge

Note : le démarrage après restart peut être capturé sur un front descendant du reset mais les temps de premier comptage et d'allumage de la LED sont trop long pour être capturés sur la fenêtre de capture de l'ILA. La validation sera faire avec l'oscilloscope

- Validation du changement de couleur de la LED sur l'ILA



On choisit de vérifier le passage de vert à rouge.

On trigge sur le signal led_g sur front descendant, on le déclenche lorsque la LED s'allume en vert pour la troisième fois pour capturer le moment où elle s'éteint

=> la LED verte d'éteint led_g = 0

=> le compteur de cycle passe à trois puis est remis à 0 au coup d'horloge suivant, nb_cycle= 2 puis 3 puis 0

Nous ne pouvons pas vérifier l'allumage de la LED rouge car les délais de 2 secondes ne peut pas être capturé par l'ILA

Note afin de réaliser plus facilement les mesures sur l'oscilloscope, j'ai rajouté des ports de recopie des sortie led_r, led_b, led_g pour pouvoir les router sur des port du connecteur J1

Voici les modification faites sur le fichier source tp_fsm.vhd: rajout d'un signal interne sur les led et double routage sur les deux port pour chaque LED.

```
entity tp_fsm is
    generic (
        cible : integer := 20000000
    );
    port (
        clk      : in std_logic;
        reset    : in std_logic;
        restart   : in std_logic;
        led_r     : out std_logic;
        led_b     : out std_logic;
        led_g     : out std_logic;
        port_led_r : out std_logic;
        port_led_b : out std_logic;
        port_led_g : out std_logic
    );
end tp_fsm;

architecture behavioral of tp_fsm is

    type state is (idle, blanc, rouge, bleu, vert); -- a quoi sert le IDLE?
    signal current_state : state; --etat dans lequel on se trouve actuellement
    signal next_state : state; --etat dans lequel on passera au prochain coup d'horloge
    signal resetn : std_logic := '0';
    signal end_count3 : std_logic := '0'; -- detection de 3 cycles comptés
    signal cpt_raz : std_logic := '0'; -- remise à zér du compteur de cycle
    signal nb_cycle : std_logic_vector (1 downto 0);
    signal led_drive : std_logic := '0';
    signal in_led_r : std_logic := '0';
    signal in_led_b : std_logic := '0';
    signal in_led_g : std_logic := '0';
begin
```

```

in_led_r <= led_drive when current_state = rouge or current_state = blanc
    else '0';
in_led_b <= led_drive when current_state = bleu or current_state = blanc
    else '0';
in_led_g <= led_drive when current_state = vert or current_state = blanc
    else '0';

port_led_r <= in_led_r;
port_led_b <= in_led_b;
port_led_g <= in_led_g;
led_r <= in_led_r;
led_b <= in_led_b;
led_g <= in_led_g;

```

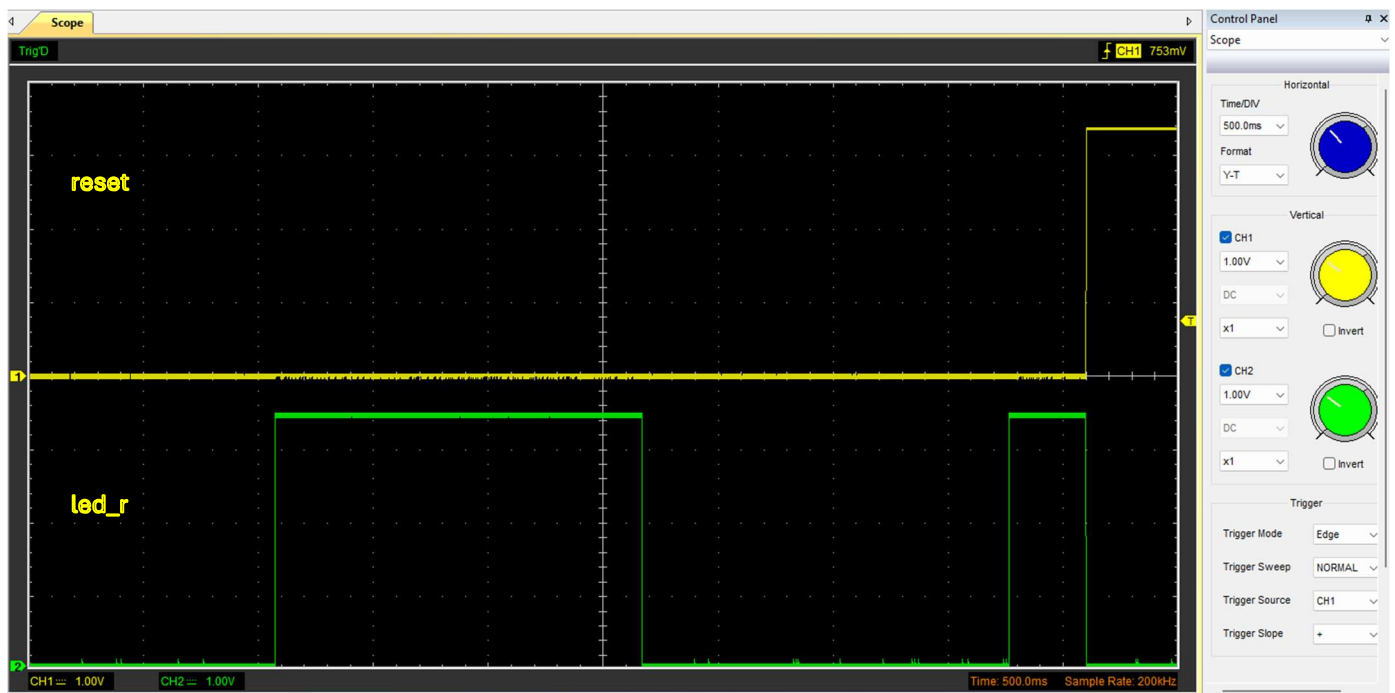
modification du fichier de contraintes pour rajouter la connexion aux port de J1

```

## User Digital I/O Header J1
set_property -dict {PACKAGE_PIN L19 IOSTANDARD LVCMOS33} [get_ports port_led_r]
##set_property -dict { PACKAGE_PIN M19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[2] }]; #IO_L7P_T1_AD2P_35 Sch=user_dio[2]
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports port_led_b]
##set_property -dict { PACKAGE_PIN P20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[4] }]; #IO_L14N_T2_SRCC_34 Sch=user_dio[4]
set_property -dict {PACKAGE_PIN P19 IOSTANDARD LVCMOS33} [get_ports port_led_g]
##set_property -dict { PACKAGE_PIN R19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[6] }]; #IO_0_34 Sch=user_dio[6]
set_property -dict { PACKAGE_PIN T20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[7] }]; #IO_L15P_T2_DQS_34 Sch=user_dio[7]

```

- Validation du reset sur l'oscilloscope

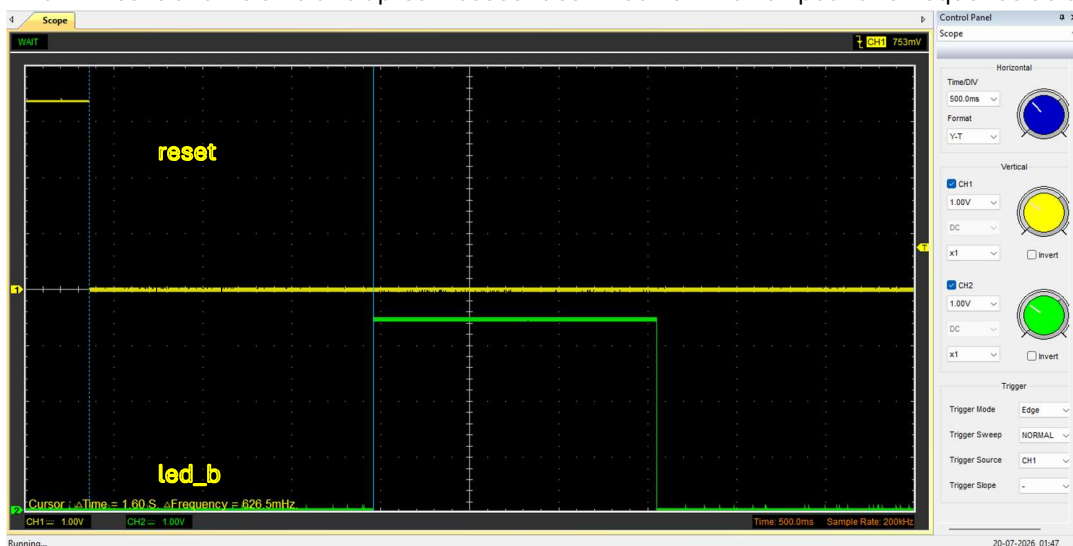
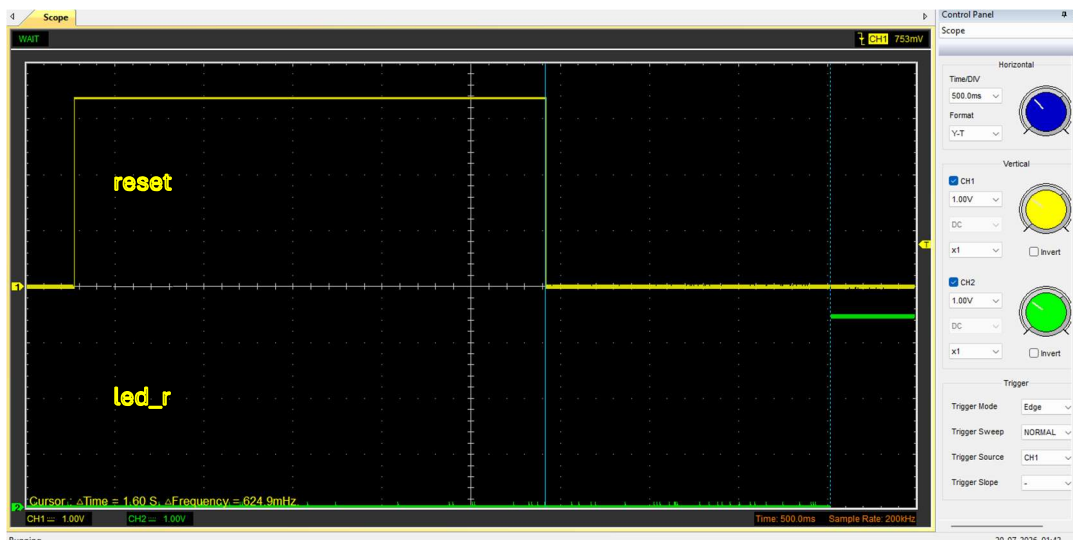


On trigge sur le signal reset sur font montant

Le bouton reset (BTN0) est appuyé et maintenu sur le deuxième cycle d'allumage de la LED rouge

=>la LED est éteinte immédiatement led_r

- Validation du démarrage après reset sur l'oscilloscope



Même principe , on mesure la led verte sur le channel 2
le led verte démarre 1.6sec après le front descendant du reset

La led démarre avec ses trois couleur actives 1.6 secondes après avoir relâché le reset , elle sera donc blanche

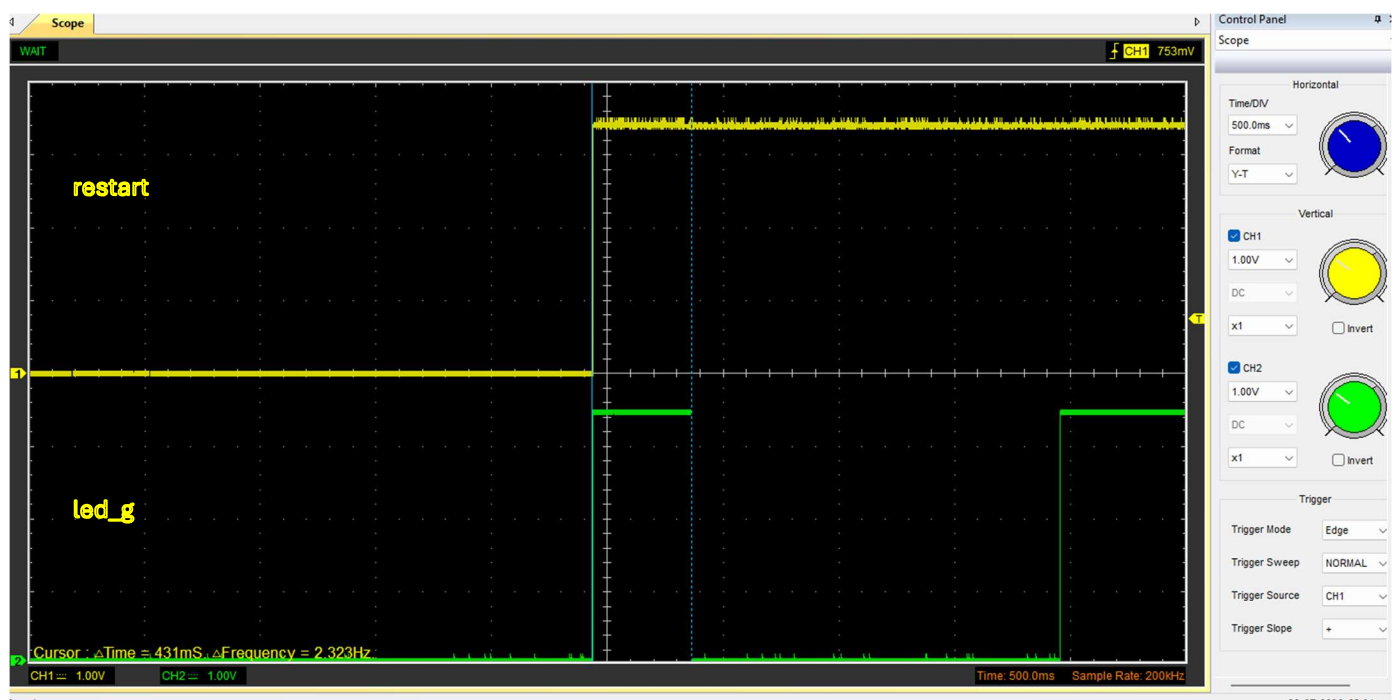
- Validation du reset sur l'oscilloscope



On trigge sur le signal reset sur font montant

Le bouton reset (BTN0) est appuyé et maintenu sur le deuxième cycle d'allumage de la LED rouge
=>la LED est éteinte immédiatement led_r

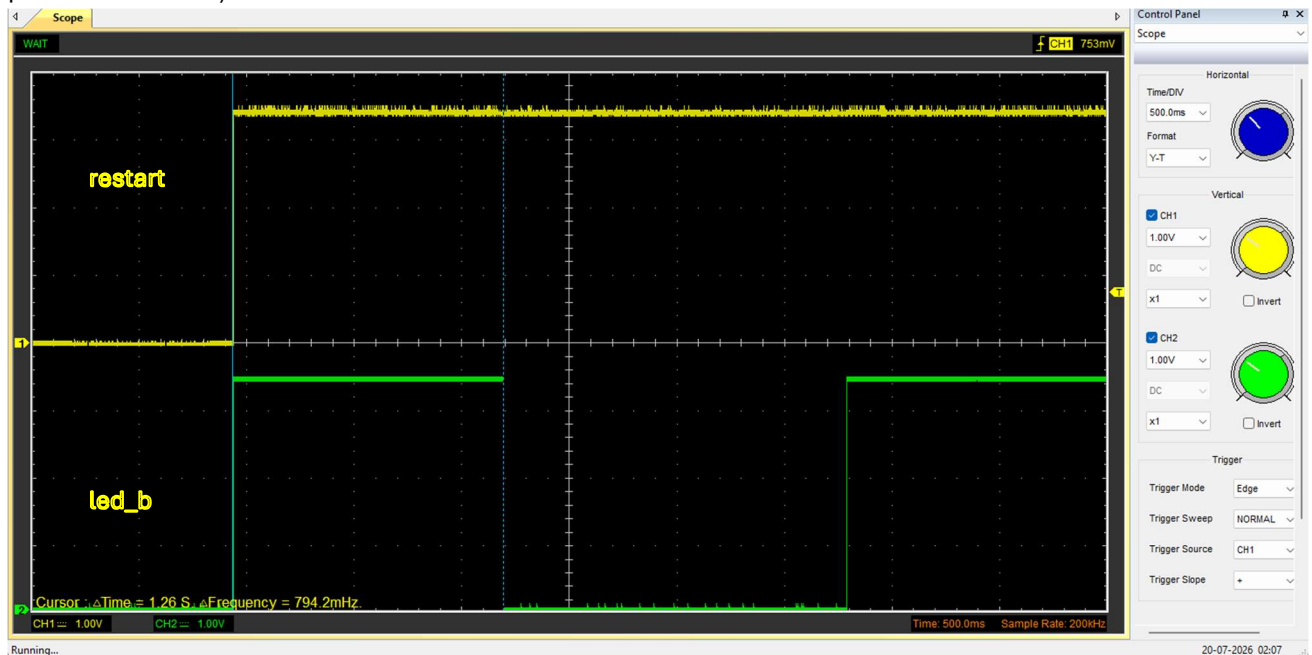
- Validation du restart sur l'oscilloscope



On trigge sur le signal restart sur font montant

Le bouton restart (BTN1) est appuyé et maintenu sur le deuxième cycle d'allumage de la LED rouge

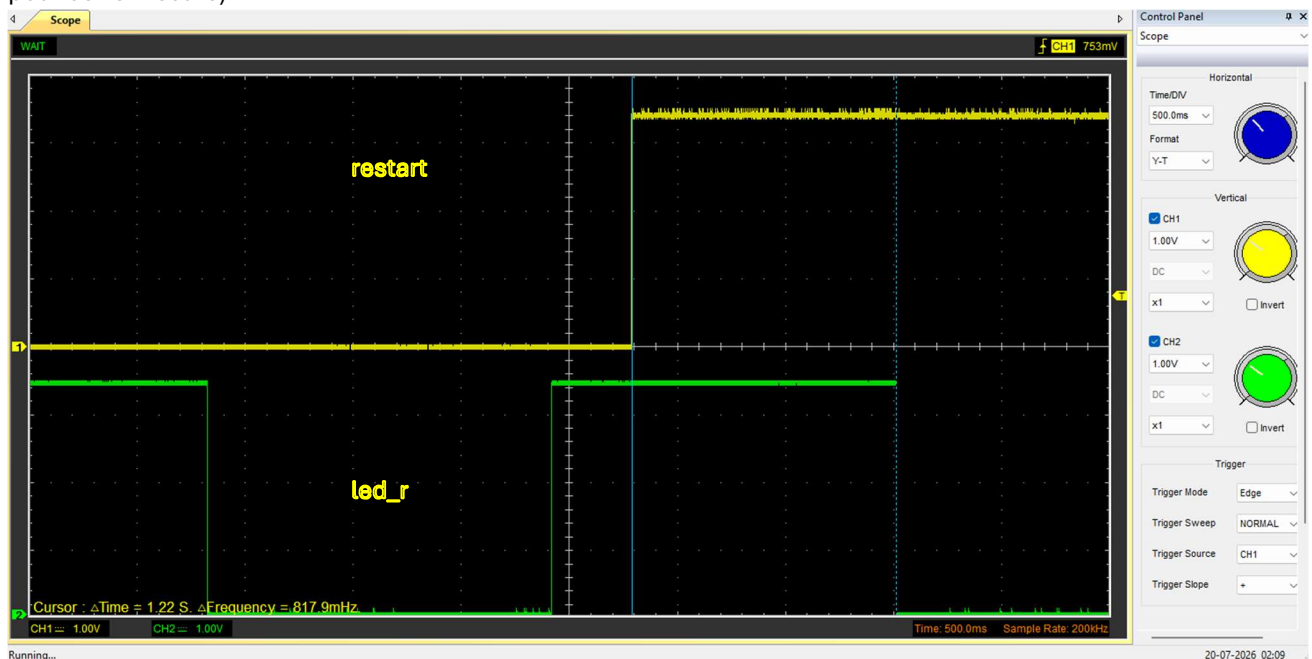
=>la LED s'allume en blanc donc la led_g s'allume sur la durées restante du cycle d'allumage de la LED (431ms pour cette mesure)



On trigge sur le signal restart sur font montant

Le bouton restart (BTN1) est appuyé et maintenu sur le deuxième cycle d'allumage de la LED rouge

=>la LED s'allume en blanc donc la led_b s'allume sur la durées restante du cycle d'allumage de la LED (1.26s pour cette mesure)

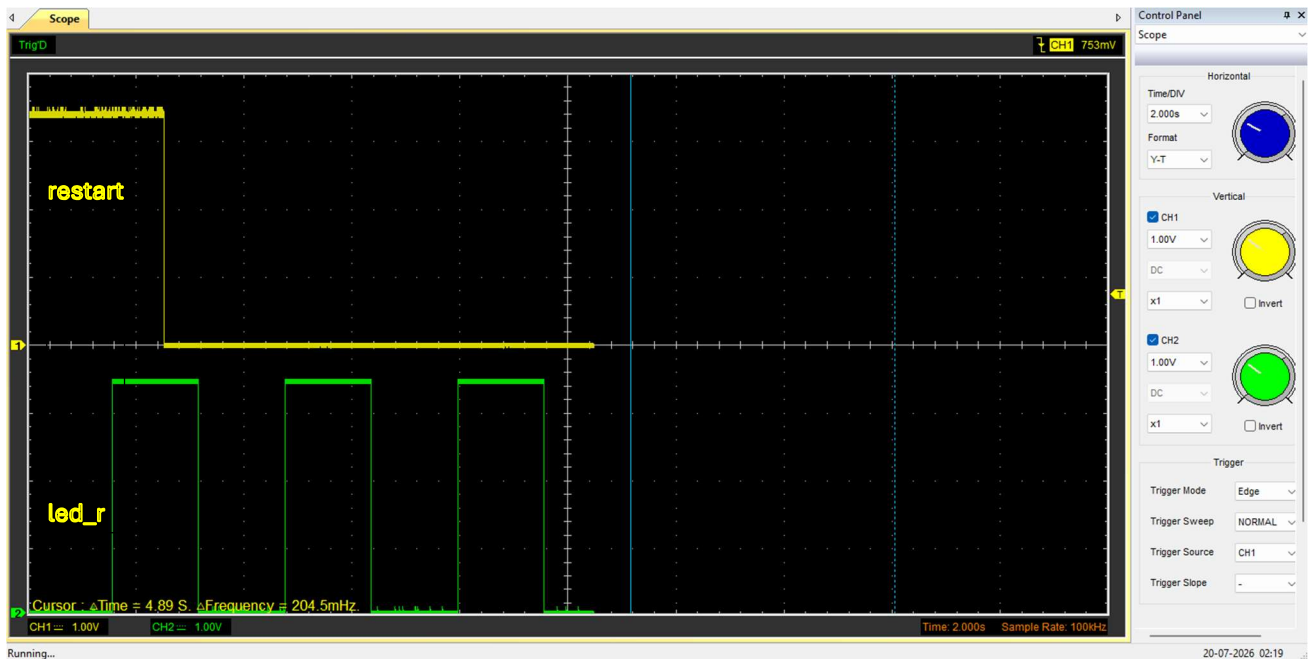


On trigge sur le signal restart sur font montant

Le bouton restart (BTN1) est appuyé et maintenu sur le deuxième cycle d'allumage de la LED rouge

=>la LED s'allume en blanc donc la led_r reste allumée sur la durées restante du cycle d'allumage de la LED (1.226s pour cette mesure)

- Validation du redémarrage du restart sur l'oscilloscope



On trigge sur le signal restart sur front descendant

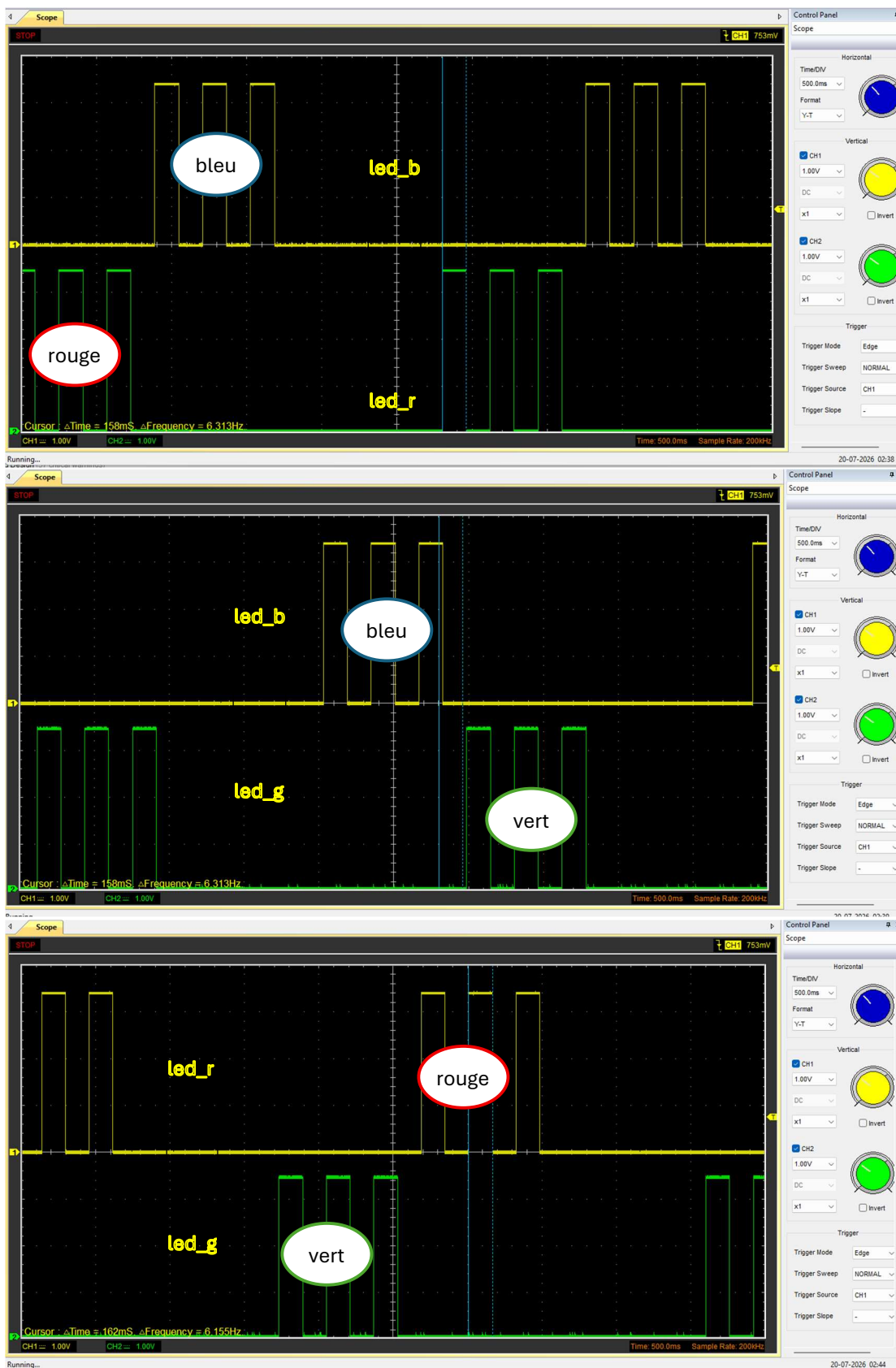
Le bouton restart (BTN1) est relaché

=>la LED continue à clignoter en blanc donc la led_r reste allumée sur la durées restante du cycle d'allumage de la LED

On a les même mesures sur les ports led_b et led_g

Il faudrait pouvoir mesurer sur une durée supérieure à 3 cycles après avoir relâché le restart pour vérifier que l'on passe bien à un cycle rouge après 3 blanc. Nous sommes bloqué par la capacité du mesure du handtek qui ne permet pas de mesurer sur des longues durées >6 secondes pour pouvoir visualiser plus de trois cycles d'allumage. Solution : réfaire la synthèse et l'implémentation et la programmation du fpga avec un temps d'allumage de 160ms soit une cible = 20 000 000 avec une fréquence sys_clk = 125Mhz

- Validation des cycles sur oscilloscope avec un temps d'éclairage de 160ms



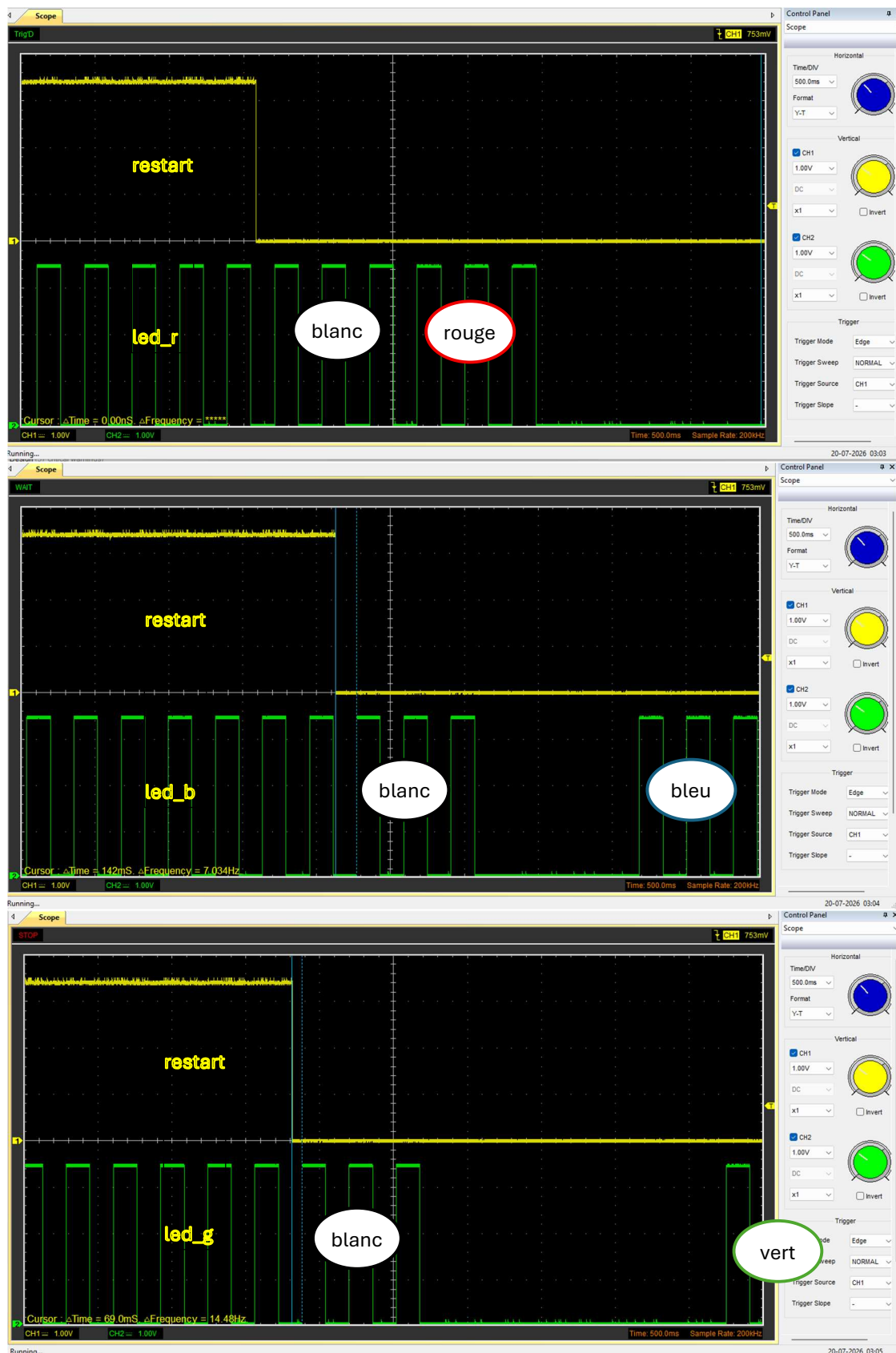
Sur ce s2 captures , on peut vérifier le cycle rouge => bleu => vert=>rouge

- Validation du démarrage après reset avec un temps d'éclairage de 160ms



Après avoir relâché le reset: les trois couleur sont allumées sur 3 allumage puis uniquement le rouge, puis uniquement le bleu puis uniquement le vert

- Validation du démarrage après restart avec un temps d'éclairage de 160ms



Pendant le restart, la LED clignote en blanc.

Quand on relâche le restart, nous avons un cycle de 3 allumage blanc puis rouge, puis bleu puis vert puis rouge ...

Conclusion :

Le FPGA fonctionne correctement suivant les spécifications de l'énoncé.